These notes are a bit of a mess. The mathematical/analytical models are not really well done, but they shouldn't be asked at the exam.

- Net
  Split Horizon
  Spanning Tree
- App
  DNS
- Security
  RSA and SSL

1) Introduction And Fundamentals

First let's recall some useful mathematical results:

$$\sum_{i=1}^{\infty} ip^i = \frac{p}{(1-p)^2}$$

1.1) ISO/OSI

ISO is the International Organization for Standardization that agrees on standards
OSI is the Open System Interconnection that defines the seven layers and it's protocols

What is a **protocol?**
A protocol is a series of steps, involving 2+ parties, that is designed to accomplish a task. In a network everyone must know the protocol. It must be **unambiguous and complete.**
Todays technology uses a divide and conquer (layered) approach. This reduces complexity, is less confusing and allows **encapsulation** at the cost of a loss in efficiency.

in the 70', the International Organization for Standardization (ISO) proposed the Open System Interconnection (OSI) protocols with seven layers. Each layer sends data above, and there is dialogue only on same level

Layer Name	Description	Examples
Application	User Level Processing	Telnet, FTP, Mail
Presentation	Data Representation & Syntax	ISO Presentation
Session	Sync Points and Dialogs	ISO Session
Transport	Reliable End to End	TCP
Network	Unreliable Thru Multi-Node Network	X.25 Pkt, IP
Link	Reliable Across Physical Line	LAPB, HDLC
Physical	Unreliable Wire, Telco Line	RS232, T1, 802.x

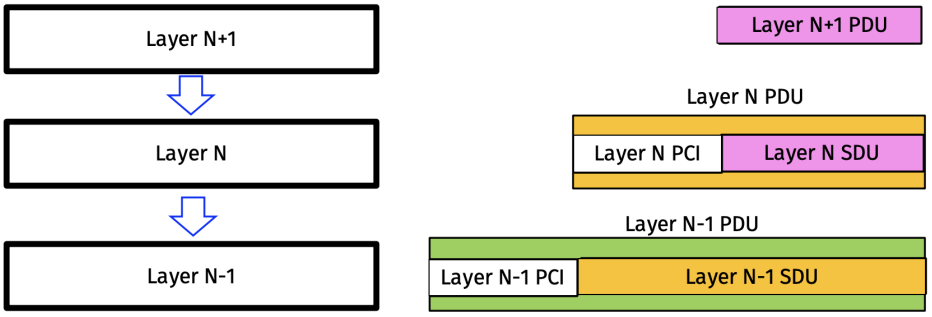
OSI Layers

- Physical: data over physical medium. Takes bits from Link layer and sends them as physical signals. They are embedded with header and tail
- Link: delivers packets between 2 nodes on a local network. Has 2 sublayers
  - Medium Access Control (MAC): avoids collision between multiple users on same network
  - Logical Link Control (LLC): flow and error control on packets
- Network Layer: This is responsible for host to host routing and forwarding of data. It uses Internet Protocol (IP) address
- Transport: offers process to process communication, via the use of ports

Each layer can communicate through frames. Data on different layers must encapsulate/decapsulate to "understand" the data.
We define

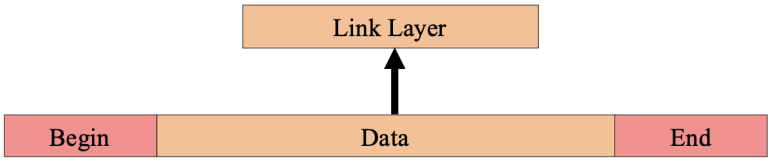
- **PDU** protocol data unit: defines the dialogue between two equal layers
- **SDU** service data unit: is the PDU of lower layers
- **PCI** protocol control information, header: is the data to make the new PDU

The lower you go, the more data you have, for example:



Example

In general, a variation of such a scheme is valid:



Example

1.2) Performance Analysis

Keep in mind that:

$$\boxed{1\ [\text{byte}] = 8\ [\text{bits}]}$$

The main performance metrics for **traffic** are:

Name	Symbol	Unit	Explanation
Bandwidth	W	Hz	Range of frequencies contained in modulated signal, or range that a channel can acceptably pass
Bitrate	R_0	bps	*Maximum* transmission speed of a link at bit level
Throughput	S	$bit/s,\ pck/s,\ pck/slot$	This shows the average rate at which user information units (PDU) are successfully delivered. It depends on the time interval. Moreover $S \leq R_0$
Goodput	S_g	$bit/s,\ pck/s,\ pck/slot$	throughput scaled by overhead and other inefficiencies. $S_g = S \cdot \frac{SDU}{PSU} \leq S$

The main **delay** metrics are:

Name	Symbol	Unit	Explanation	Formula
End-to-End Latency (Delay)	d_{e2e}	s	overall time it takes to deliver a complete message	$d_{e2e} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$ $\approx d_{prop} + d_{trans}$
Processign Delay	d_{proc}	s	time it takes the receiver to process the packet. Often it is calculated as an avg and is non negiglible	
Queueing Delay	d_{queue}	s	time packet waits in i/o queues in router	
Transmission Delay	d_{trans}	s	Time it takes to send first to last bit	$d_{trans} = data/R_0$
Propagation Delay	d_{prop}	s	time it takes to send a bit and to have it recieved	$d_{prop} = distance/speed$ usually $speed = c = 3 \cdot 10^8$

Name	Symbol	Unit	Explanation	Formula
Jitter	J	s	variation in latency between packets	
Round Trip Time	RTT	s	Time it takes from the moment i send first bit until ack	$RTT_{min} = d_{trans-AB} + d_{prop-AB} + d_{prop-BA}$
Bandwidth Delay product (capacity)	BDP	$bits$, $pcks$	maximum long term average number of bits that can be transferred in an e2e interval	$BDP = RTT \cdot R_0$

The **efficiency** is defined as

$$\eta = \frac{\text{data sent}}{\text{capacity}} = \frac{T_{TX}}{RTT_{min}}$$

In this case the capacity is the time we wait between sending packets

1.3) Network Architectures

The following table shows the difference in the 3 types of networks by scope:

Parameter	LAN	MAN	WAN
Ownership of Network	Private	Private or Public	Private or Public
Design and Maintenance	Easy	Difficult	Difficult
Propagation Delay	Short	Moderate	Long
Speed	High	Moderate	Low
Congestion	Less	More	More
Application	College, School, Hospital	Small towns, City	Country/Continent

Table

Three more niche are WLAN, WSN, BAN.

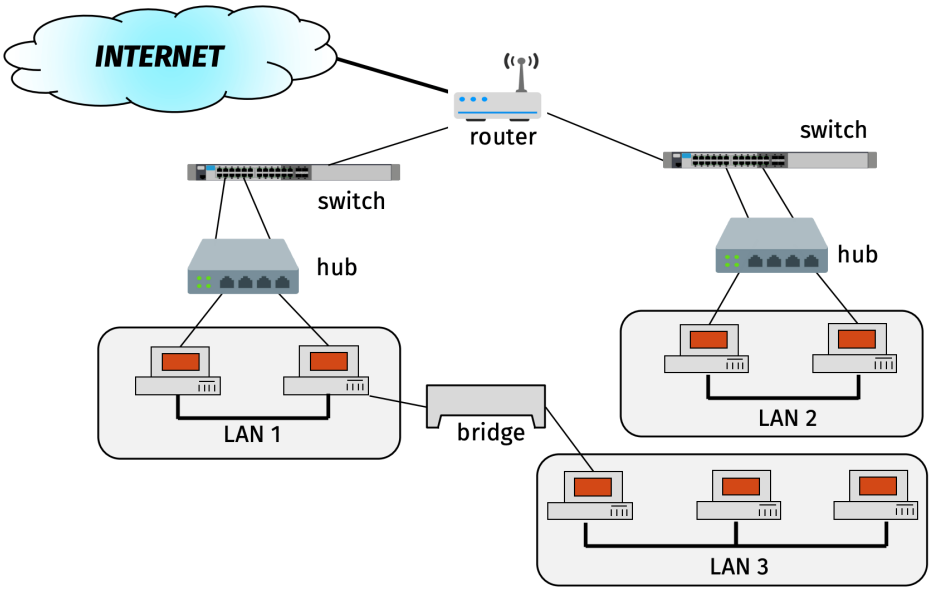
There are various transmission mediums: Air, Optical Cables, Copper Cables, VLC (visible light communication)

Network Topologies

- **Bus**: Every node taps in a common medium. Direct communication between all nodes possible at risk of higher collision probability
- **Star:** There is one central (master) node which connects every other (slave) device. Used for small networks (LAN)
- **Ring:** Arranged in a ring. Every node acts as a repeater
- **Mesh:** Every node directly connects to every other node, it needs $n(n-1)/2$ links

Network Elements

Suppose to have the following configuration:



Configuration

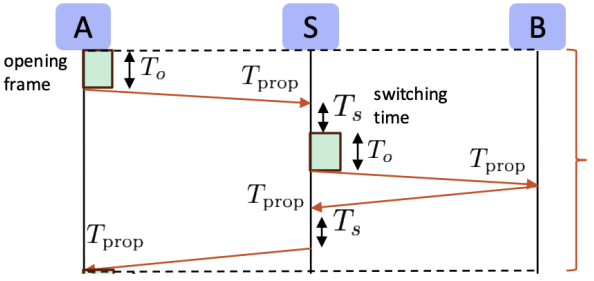
- **Hub:** is a physical layer (L1) (no MAC) and acts as repeater (start topology). It doesn't know what port to send the signal, it is broadcast
- **Bridge:** data link layer (L2), divides network in 2 (max 4) subnets. It manages the traffic flow. It can be used to block many packets that aren't supposed to enter that LAN
- **Switch:** both physical and data link (L2). Is a hub that knows what port to send the packets to.
- **Router:** Network layer device (L3). Connects LAN to internet.

1.4) Switch

The switch operates on DLL layer (2). It manages MAC addresses in a dynamic table in order to avoid sending each message as broadcast but directly in correct port. There are 3 operational switching modes:

Circuit Switched (CS) Network

Before communication, establish physical connection between 2 devices. To establish the connection a packet is sent that "maps" the correct path. The connection establishment time is $T = N(t_0 + 2t_p + 2t_s) + t_0 + 2t_p$ usually t_0 and t_s are neglected, therefore if there are N switches the connection time is $T = 2(N + 1)t_{prop}$.



Example N=1

Datagram Packet Switched (PS) Network

The data is divided in packets that independently get delivered. The intermediate node **stores and forwards** the packet. different paths may lead to ooo packets. Since we have no delay in RTT the slowest (bottleneck) link will store the packets in queue. This link will be the main link to calculate the delivery time, as it will process the packets the slowest. With N links and N_p packets:

$$T = (N_p - 1) \cdot \max(T_{tx}) + \sum T_p + \sum_1^N T_{tx}$$

Moreover, given the time to deliver the first packet $T_1 = \sum T_p + \sum_1^N T_{tx}$, the total delivery time is $T_{N_p} = (N_p - 1)\max(T_{tx}) + T_1$

Virtual Packet Switched (PS) Network

This is in between the previous two methods. We first establish a connection, then we send packets. We cannot end up with ooo packets.

2) Datalink Layer (DLL)

The DLL must do the following:

- Framing: encapsulation
- Link Access (MAC): shared medium
- Flow Control (over a link):
- Error Detection: ARQ
- Error Correction: correct code without needing retx

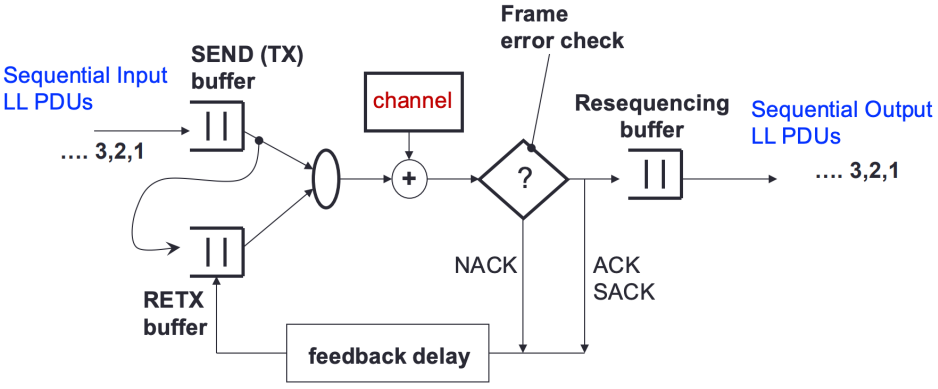
2.1) Data Link Control (DLC)

2.1.1) Error Control

Suppose we can detect errors on received packages. We can make the channel reliable by using **Automatic Repeat reQuest (ARQ)** algorithms. These use a special type of packet sent by the receiver side called **acknowledgment (ACK)** in order to let the sender know if there have been errors. ARQ manages to **make the channel *errorless/accurate* and *efficient***

Types of ACKs:

- Selective (S)ACK(n): specifies what packets were accepted. n-th frame correctly received
- Cumulative ACK(n): all packets have been received. Expects n-th frame
- Negative (N)ACK(n): refusal to accept packet. OOO, expects n-th frame for in order



ARQ Diagram

Some situations might cause the ack to never be received, they give raise to 2 main problems:

- If the frame is never received there won't be any ACK generated and we end up in **deadlock**, a *stalling* position where the sender waits for and ACK to TX but won't. A solution is the **ACK time-out** that RETX n-th frame if ACK(n) is missing, different RETX strategies can be more efficient.
- If the frame arrives but the ACK is lost we end up sending (after time out) a **duplicate packet**. This is easily solved by implementing **sequence numbers (SN)** where te receiver discards packets with same sequence number

Before studying specific protocols, let's explicit our definitions and key assumptions:

- **Definitions:**

Param	Description	Param	Description
R_{LL}	bitrate at the LL [bit/s]	ρ	channel utilization factor $\in[0,1]$
F	(H+I) frame size [bits]	η	normalized goodput efficiency
t_F	F/R_{LL} frame TX time [s]	τ_p	propagation delay [s]
t_I	TX time (DATA only) [s]	t_A	A/R_{LL} ACK TX time [s]
t_T	total frame TX time [s]	RTT	time to TX and ACK a pkt [s]

Definitions

clearly we have that $I = t_I R_{LL}$ and $F = t_F R_{LL}$.

- **Key assumptions:**
  We assume that our send buffer is never empty, that is, the channel always filled with TX frames. This allows to obtain upper bounds for efficiency
  Keep in mind that $t_F = t_H + t_I$ is the sum of the header + the data

	S&W	**GBN**	**SR**
Efficiency	ineffficient if $F < R_{LL}RTT \implies t_F << RTT$	continuous transmission if $t_o = Nt_F \geq RTT$	
$E[t_T]$	$\frac{RTT}{1-p} = \frac{t_F + t_A + 2\tau_p}{1-p}$	$t_F + RTT\frac{p}{1-p}$	$\frac{t_F}{1-p}$
Utilization ρ	$\frac{t_F(1-p)}{RTT}$ never goes to 1	$\frac{F(1-p)}{F + (A + 2R_{LL}\tau_p)p}$	$1-p$
Goodput η	$\frac{I}{F}\rho$		$\frac{I}{D}(1-p)$

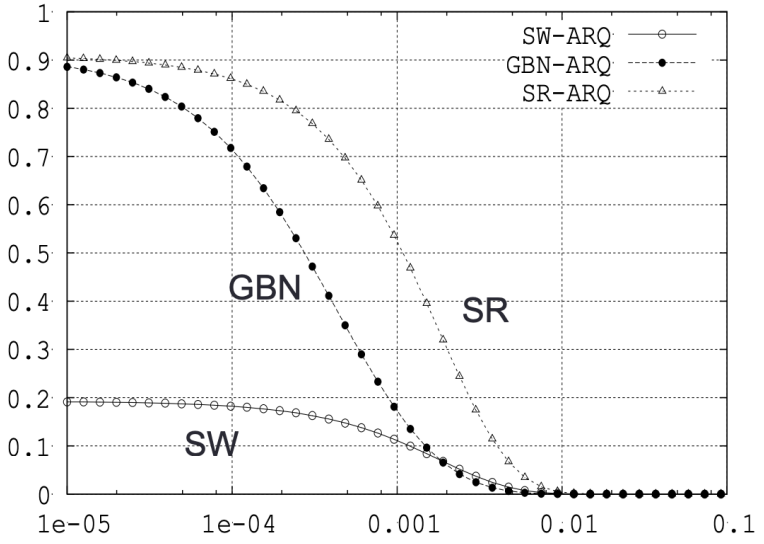
ARQ performance depends on packet size. Shorter packets are better on channels prone to error.

In the case of SR it is possible to determine the ideal packet size by looking at the throughput. Set $H := o$, $I + H := x$ and recall that $1 - p = (1 - P_{bit})^x$

$$g = R_{LL}\eta = R_{LL}\frac{x-o}{x}(1-P_{bit})^x \xrightarrow{\frac{d}{dx}} R_{LL}\frac{x(x-o)\ln(1-P_b) + o}{x^2}(1-P_b)^x$$

from here $x = \dfrac{o \pm \sqrt{o^2 - \frac{4o}{\ln(1-P_b)}}}{2}$

S&W is used for small pipelines as it performs optimally for pipe capacity of 1, so it is common at the DLL. TCP uses GBN



Stop & Wait ARQ (SW-ARQ), IEEE 802.11

Really simple:

1. Send frame i
2. Wait for ACK or TO
3. If ACK then i++, otherwise keep i. Restart from 1)

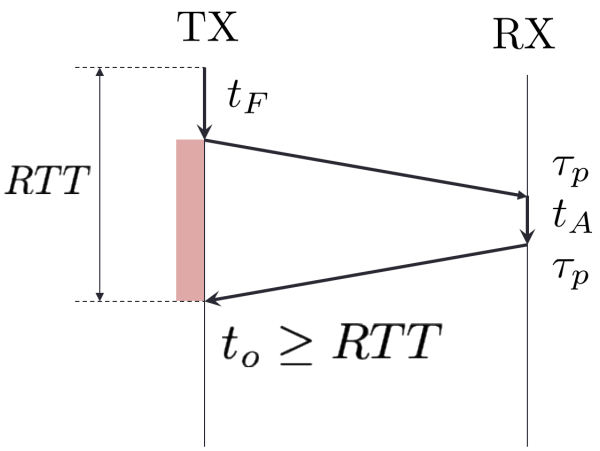
In this case

$$RTT = t_F + 2\tau_p + t_A$$

The expected time to correctly transmit is given by

$$E[t_T] = \frac{RTT}{1-p}$$

This is given by: $E[t_T] = RTT + \sum t_i P_i$ where $t_i = i \cdot RTT$ and $P_i = p^i(1-p)$



Diagram

Clearly t_i is the total RETX time and P_i is the probability to have i RETX (also clearly a geometric series). By keeping in mind that $\sum_0^\infty ip^i = p/(1-p)^2$ the result of the expected value is straight forward.

Moreover

$$\rho = \frac{t_F}{E[t_T]} = \frac{t_f(1-p)}{RTT} \overset{p\to 0}{\to} \rho = \frac{t_f}{RTT}$$
$$\eta = \frac{t_I}{E[t_T]} = \frac{t_I}{t_F}\frac{t_F}{E[t_T]} = \frac{I}{F}\rho$$

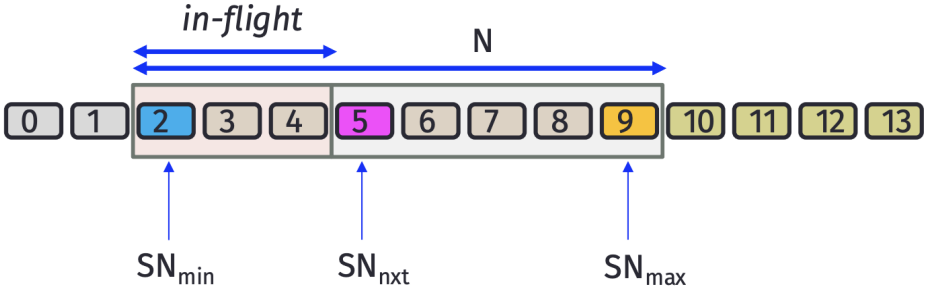
SW-ARQ is **inefficient if** $F << R_{LL}RTT \iff t_f << RTT$

This implementation can send only one frame per RTT and has capacity $R_{LL}RTT$.

Go-Back-N (GBN)

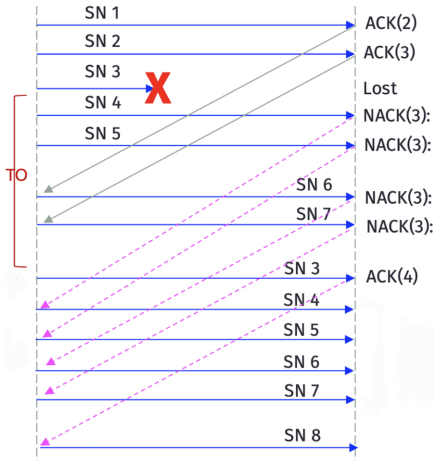
A **sliding window protocol** transmits **multiple unacknowledged** frames **back to back.**

- On the sender side the window (sendW) is the number of *unacknowledged* frames that che can TX over the channel. It may grow and sink and moves on ACKs.
  Usually we have that $SN_{min} = ACK(n)$



On the receiver side we have a window of size M of packets that can be received

Go-Back-N sends all N pkts. $SN_{min} = ACK(n)$ and sends until pkt $n + N - 1$.
If a pkt is lost we receive NACK(n) and after TO it RETX n. The receiver discards all packets that are ooo



GBN

To have continuous RETX we must have

$$t_o \geq t_f + 2\tau_p + t_A = RTT$$

$$NT_F \geq RTT$$

Here we have:

$$E[t_T] = t_F + \sum t_i P_i = t_F + RTT\frac{p}{1-p} = \frac{t_F + (t_A + 2\tau_p)p}{1-p}$$

$$\rho = \frac{t_F}{E[t_T]} = \frac{t_F(1-p)}{t_F + (t_A + 2\tau_p)p} = \frac{F(1-p)}{F + (A + 2R_{LL}\tau_p)p} \overset{p\to 0}{\to} \rho = 1$$

Selective Repeat ARQ (SR-ARQ) IEEE802.11ax

This is a smarter GBN protocol since the receiver has a queue where the OOO packets are saved until they are in order and can be grouped. There is no NACK, but a SACK. The SACK doesn't move the SNmin and the packet gets resent only after TO.

$$E[t_T] = \frac{t_F}{1-p}$$

$$\rho = \frac{t_F}{E[t_T]} = 1 - p \approx 1 - (I + H)P_{bit}$$

$$\eta = \frac{I}{F}(1-p) \approx \frac{I}{I+H}(1 - (I+H)P_{bit})$$

2.1.2) Framing

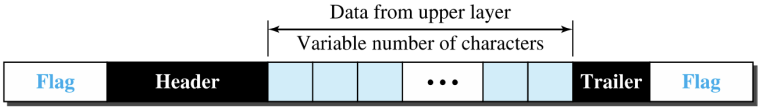
As we have seen, we don't send the entire information in one go, but we send a frame made of header+part of information.

Usually the frame is of variable size, either:

- Character Oriented: send multiples of 8 bits (1 byte)
- Bit Oriented: no constraint

With fixed size frames there are no issues, since we can count the buts to know when a frame starts/ends.

The prevalent framing method however is of variable size and therefore needs a flag to be delimited



Example

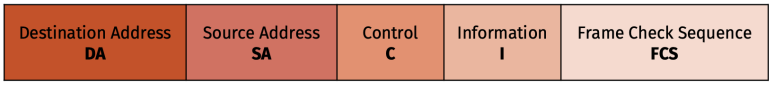
We define a special character called **flag** to delimit the frame [01111110]. To ensure that this is not mistaken with a part of the message, we use **bit stuffing** and add a 0 before the last 1 in the message. This is done recursively (?)

2.2) Medium Access Control (MAC)

This layer is responsible for the delivery of PDU between source and destination and resolves conflicts between multiple access to the channel

2.2.1) Random Access

All stations are equal and there is a contention process given by random values to allow for transmission. If more than one station sends simultaneously there is a collision and the data of both are lost.



PDU

2.2.2) Aloha Protocols

	Pure Aloha	**Slotted Aloha**
Throughput $S = GP_S$	$S = Ge^{-2G}S_{\max} = 18\%$	$S = Ge^{-G}S_{\max} = 36\%$
Vulnerable time	$2t_F$	t_F
Offered Traffic	$G = \lambda' t_F$	
Avg transmissions	$E[n_{tx}] = 1/P_S$	

On average slotted aloha has a higher throughput than pure aloha. For $G \leq 10^{-1}$ slotted aloha has a bigger delay.

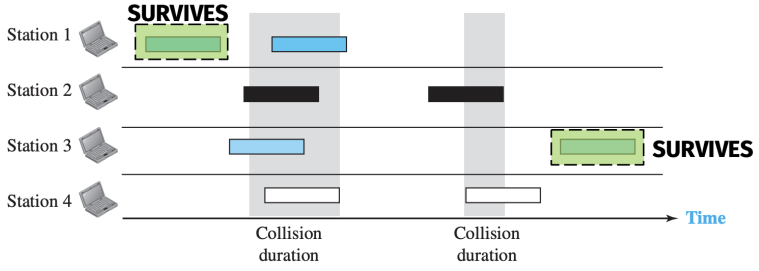
Recall that:

- for smaller prop. delays CSMA performs better
- for bigger prop. delays Slotted Aloha performs better

So CSMA is for short range communication, while aloha for long range (it was intended to work with satellites)

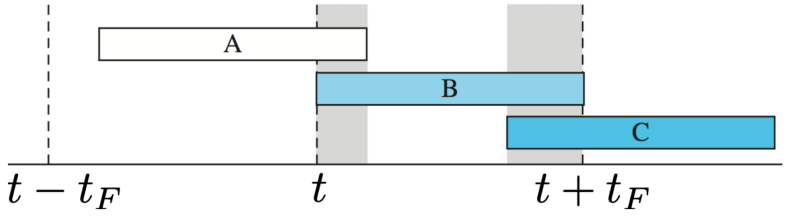
(Pure) Aloha

A packet survives only if there is no collision during the entire transmission. Aloha protocol makes packets wait a random backoff time before a RETX after a collision.



Aloha

The **channel vulnerability period** is $\boxed{2t_F}$ since we must be sure that t_F seconds before the transmission and the t_F seconds of the transmission the channel is empty.



channel vulnerability period

Offered Traffic
This is the avg number of frames that arrive in a service time. Define λ' as new+collided frame service time:

$$G = \frac{\lambda'}{\mu} = \lambda' t_F$$

Success Probability
is the probability of a successful transmission

$$P_S = \frac{\lambda}{\lambda'}$$

Throughput
this is the avg number of succesfully TX packets

$$S = GP_S = \lambda t_f = Ge^{-2G}$$

If we find the maximum of the throughput we end up with $S_{max} = \frac{1}{2e} \approx 0,18$

The **avg number of tx is**:

$$E[n_{tx}] = \sum_{1}^{\infty} n(1 - P_s)^{n-1} P_S = \frac{1}{P_S}$$

Then we can set the **avg number of retransmissions** as

$$E[n_{retx}] = E[n_{tx}] - 1 = \frac{1 - P_S}{P_S}$$

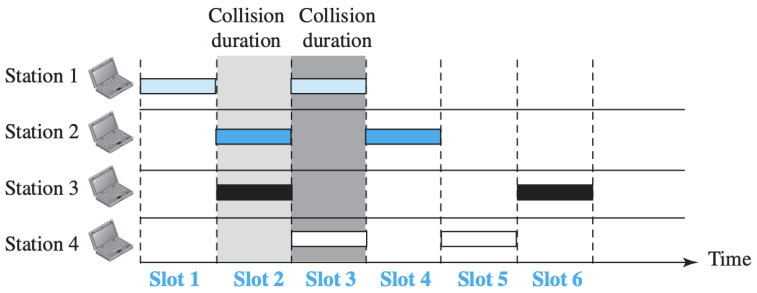
From here we can set the delay as the sum of the last transmission $t_F + \tau_p$ and the retransmission $t_F + 2\tau_p + E[t_b]$ which can be rewritten

Theorem 1 (Aloha Delay)**.**

$$E[T] = (e^{2G} - 1)(t_F + 2\tau_p + E[t_b]) + t_F + t_b$$

Slotted Aloha

Like aloha, but with fixed transmit intervals. If I want to sent a packet at time t I need to wait until the slot I'm in ends.
The **vulnerability period** is t_F since it is guaranteed to send after a full TX.



Slotted Aloha

Throughput:

$$S = GP_S = Ge^{-G}$$

with max at $S_{max} = \frac{1}{e} \approx 0.37$

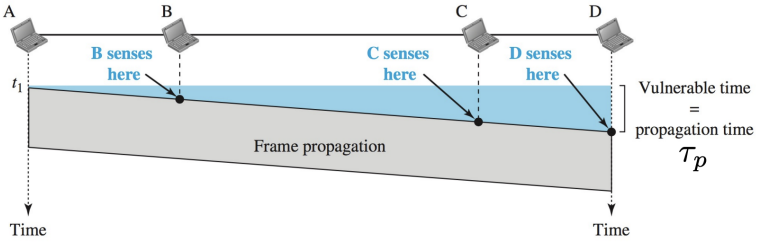
With a delay given by the last tx time $E[W] + t_f + \tau_p$ and the retx time $E[W] + t_f + 2\tau_p + E[t_b]$

Theorem 2 (Slotted Aloha Delay)**.**

$$E[T] = \frac{3t_F}{2} + \tau_p + (e^G - 1)(\frac{3t_F}{2} + 2\tau_p + E[t_b])$$

2.2.3) CSMA Protocols

Here a station sends if it senses that the channel is idle. Therefore the **vulnerability period** is $\boxed{\tau_p}$ the max propagation time.



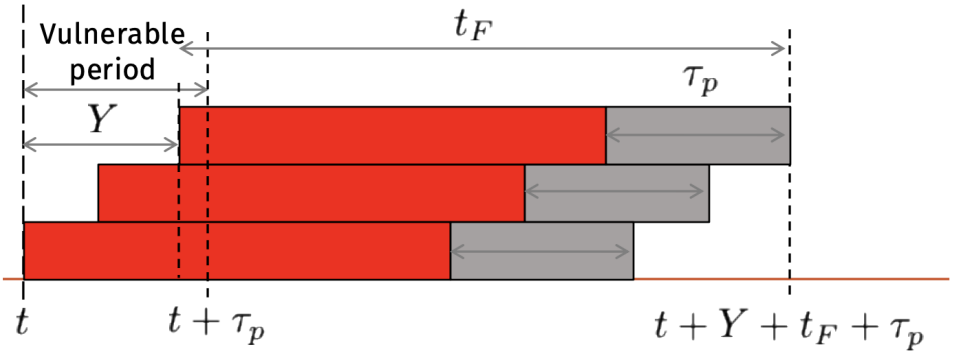
Vulnerability

There are 3 main CSMA variants:

- **1-Persistent:** TX as soon as the channel is sensed as empty
- **Non-Persistent:** TX after a random backoff time after the channel is sensed empty
- **p-Persistent:** With probability p it is 1-Persistent, with probability $1 - p$ it is Non-Persistent

Define **normalization propagation delay:** $a = \tau_p / t_F$
Also the avg idle (interarrival) period is $m_I = 1/\lambda' = t_F/G$
A BP (Busy Period) is successful if there are no collisions, that is to have no arrivals $P_{BPS} = e^{-Ga}$ and has avg. duration of $t_f + \tau_p$
the average duration of the **useful period** (correct transmission) is $t_F e^{-Ga}$

In case of collision it is a bit harder:



Time Graph

the duration of a collided cycle is $m_{BPC} = E[Y] + t_f + \tau_p$. What is $E[Y]$?
That is the expected value of the time the last collided packet is sent. After some calculations we have that $E[Y] = t_F \left(a - \frac{1-e^{-aG}}{G}\right)$

Throughput:

$$S = \frac{m_U}{m_B + m_I} = \frac{Ge^{-aG}}{G(1+2a) + Ge^{-aG}}$$

Theorem 3 (CSMA Delay)**.**

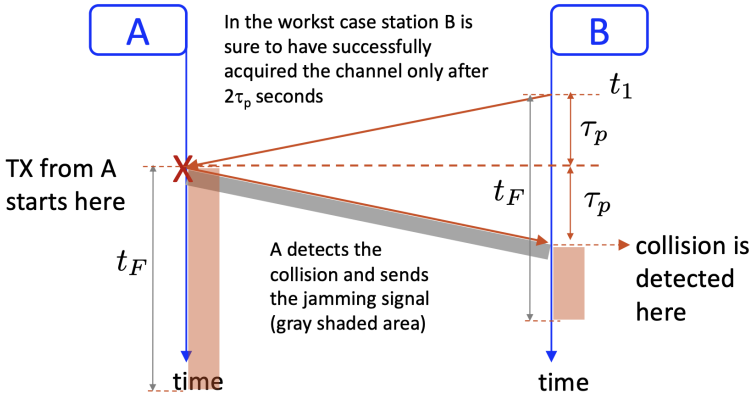
$$E[T] = E[W] + t_f + \tau_p + (\frac{G}{S} - 1)(E[W] + t_F + 2\tau_p + E[t_b])$$

Where $E[W] = \frac{1}{\beta}\frac{P_{busy}}{1-P_{busy}}$ *is the time to wait before channel is found idle and* $\frac{G}{S} - 1$ *is the # of retx failures*

CSMA/CD (Collision Detection) (Ethernet)

Here the energy levels are monitored. A collision results in an abnormal energy value. If a collision is detected the transmission is aborted and a **jamming signal** is sent to inform other stations.
In order to be able to tell if we have had any collision we must set $t_F \geq 2\tau_p$ from here

$$F_{\min} = R \cdot (t_F)_{\min} = R \cdot 2\tau_p$$



Example

CSMA/CA (Collision Avoidance) (Wi-Fi)

In this protocol the station doesn't send when channel is sensed idle, but waits a period of time called **InterFrame Space (IFS)**. Different categories of IFS exist in order to prioritize certain stations/types of communications:

- DIFS: time for channel to be idle before sending

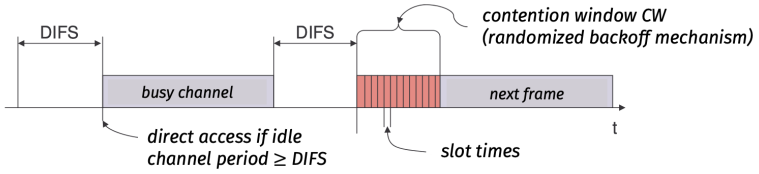
- SIFS: used for ACKs, the shortest time $= 16\mu s$ which is min separation of $2\tau_p$+local MAC processing

Moreover each station has a Backoff Timer (BT) that is frozen when channel is busy and downcounts when it is idle for DIFS seconds.

The space is actually discretized in slots. The duration of these slots depends on the Contention Window (CT) that has $CW_{min/max} = 15/1023$ and doubles every time the TX fails. Moreover the BT has a random duration in $[0, CW - 1]$. For CA it holds that $T_{slot} = 9\mu s$

It works this way:

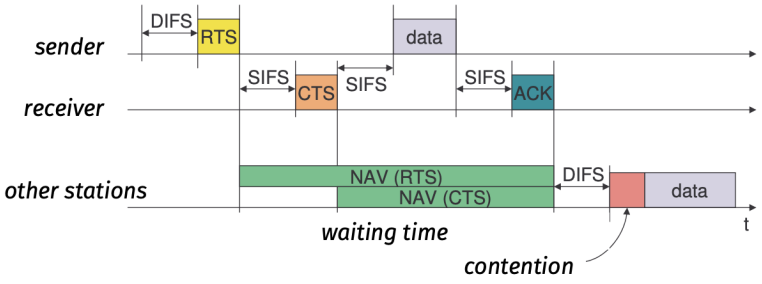
- Channel sensing is performed and data is sent after channel is idle for DIFS (protocol specific) seconds and BT has finished.
- Reciever sends ACK after SIFS seconds
- Sender uses S&W policy: sends until succesfull or max nr of retx is reached



Example

The data is not directly sent. Once the sender can TX it acquires the channel by sending a Request To Send (RTS) and the destination sends a broadcast Clear To Send (CTS). Then the data + ack communication is done.

In the meantime once the other station receive a RTS and CTS that is not directed to them they start a Network Allocation Vector (NAV) that is a timer to pause checking for idle channel. It should last until the ack is received.



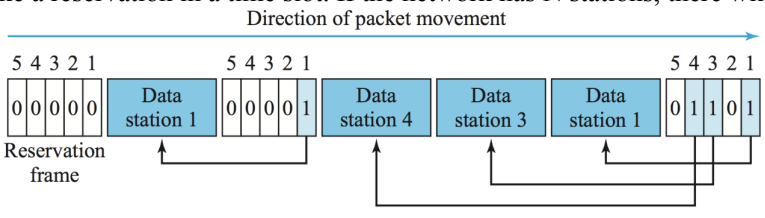
RTS CTS NAV Example

Of course collisions on the RTS or CTS might happen, so if a CTS is not received a collision is assumed.
Handshaking solves this problem: the receiver sends the CTS for the first RTS that has arrived, therefore the stations that sent a RTS and did not receive a correct CTS ill still start the NAV.

2.2.4) Controlled Access

Reservation

Every station has to make a reservation in a time slot. If the network has N stations, there will be N mini slots.



Example N=5

Polling (Bluetooth)

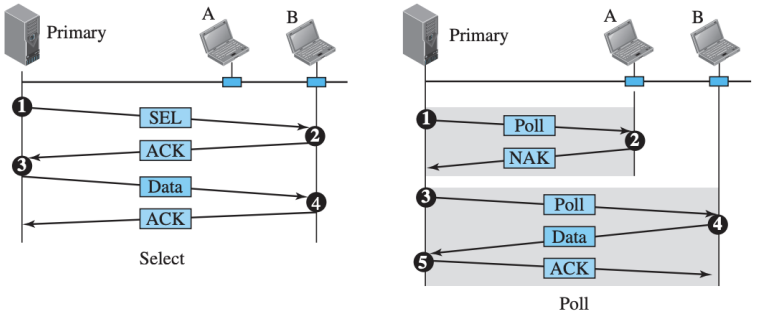
Star topology network, all data must pass through the primary node which controls the link using *poll* and *select* functions. This is a **star topology**.

Select: used by primary to send:

- primary informs the secondary of upcoming transmission using (SEL) frame and waits for ack of selected secondary station

Poll: used by primary to receive:

- primary asks (poll) every device (separately) if they have to send
- secondary responds with NACK or data
- if data is sent primary reads it and then returns ACK



Select/Poll Example

Token Passing

A token passes through the links and whoever has the token can transmit. Once a station has acquired the token it sends data (if it has to) and then passes token to the next station.
It is important to limit hold time and check if the token wasn't lost or destroyed.

2.2.5) Channelization

	TDMA	**FDMA**
Arrival Rate	$\lambda_t = N_u\lambda$	
Throughput	$S = G = N_u\lambda t_F$	$S = G = \rho = \lambda/\mu = \lambda N_u \frac{F}{R}$
Service Time	$E[y] = 1/\mu = N_u t_F$	

Moreover the normalized delay holds:

$$E[\tilde{T}]_{FDMA} = E[\tilde{T}]_{TDMA} + \frac{N_u}{2} - 1$$

where N_u is the number of users

Frequency Division Multiple Access (FDMA)

FDMA uses a predetermined frequency band, each band is reserved to a station and the sender/receiver operate with passband filters. We can see this as a M/G/1 queue.
Throughput:
This is a M/G/1 queue

$$S = \lambda N_u t_f$$

Time Division Multiple Access (TDMA)

Here the time axis is divided into slots and the transmissions go round robin. We can see this as a M/D/1 queue. The total arrival rate is $\lambda_t = N_u\lambda$, therefore:

Throughput:
This is a M/D/1 queue

$$S = G = \rho = \frac{\lambda_t}{\mu} = \frac{N_u \lambda}{\mu} = N_u \lambda t_F$$

Service Time

$$\mu = 1/N_u t_F$$

Queueing Delay

$$E[w] = \frac{S N_u t_F}{2(1-S)}$$

Delay:
The delay is given by the following 4 contributions:

1. Average slot waiting time $= N_u t_F / 2$
2. Queue waiting time $E[w] = \frac{S N_u t_F}{2(1-S)}$
3. Packet TX time $= t_F$
4. Final Propagation delay $= \tau_p$

Orthogonal Frequency Division Multiple Access (OFDMA)

This is a mix of the previous 2 cases, where time slots are used to transmit on multiple sub-bands.

Code Division Multiple Access (CDMA)

All happens simultaneously but transmissions are differentiated by a special code ???

Space Division Multiple Access (SDMA)

Here the inputs are differentiated trough beams in the spatial domain ???

2.2.6) Quick recap of QT

Arrivals
We are supposing iid interarrival times with poisson process, therefore the **mean arrival time** $E[\tau]$ defines the **arrival rate** $\boxed{\lambda = 1/E[\tau]}$
PDF and CD of poisson:

$$p_\tau(a) = \lambda e^{-\lambda a}$$
$$P_\tau(a) = 1 - e^{-\lambda a}$$

Probability to receive k customers in τ time is:

$$P(k, \tau) = \frac{e^{-\lambda\tau}(\lambda\tau)^k}{k!}$$

Service
We define the **service rate** as $\boxed{\mu = 1/E[y]}$, if there are m server it is $m\mu$
Deterministic PDF and CDF

$$p_y(a) = \delta(a - 1/\mu)$$
$$P_y(a) = u(a - 1/\mu)$$

Exponential

$$p_y(a) = \mu e^{-\mu a}$$
$$P_y(a) = 1 - e^{-\mu a}$$

A queue can hold $K = Q + m$ clients. We have a **non-blocking** QS, that is $Q \to \infty$

For stability we require $\boxed{\lambda < m\mu}$

A convenient way to specify a QS is with a **string of 6 characters** called Kendall Notation:

$$\mathrm{A/B/C/K/N\text{-}S}$$

where:

- A: specifies the statistical model of the interarrival time
- B: specifies the statistical model of the service process
- C: # of servers = m
- K: system storage capacity (default:∞)
- N: size of customer population (default:∞)
- S: service discipline (default: FCFS)

Moreover A, B can assume the following values:

- M: Memoryless/Markovian, is exponentially distributed
- D: Deterministic, constant value
- E_r: Erlang distribution with index r
- G: Generic distribution

Important measures:

- $q(t)$: # of customers waiting
- $z(t)$: # of customers in service
- $x(t)$: # of customers in whole system
- w_n: waiting/queueing time
- y_n: serving time
- s_n: system time
- $G = \lambda/\mu$: offered traffic
- r_n: interdeparture time
- $\eta = 1/E[r]$: throughput
- $S = \eta/\mu = \eta E[y] \in [0,1]$: Useful traffic; η normalized to μ
- $\rho = E[y]/me[\tau] = \lambda/m\mu = G/m$: load factor. A QS is stable if $\rho < 1$

Littles Law

Theorem 4 (Little's Law)**.**

Let the means of the number of clients, arrival rate and staying time be finite, then

$$E[x(t)] = E[\lambda(t)] \cdot E[s(t)]$$

Independently of all the assumptions on the statistical model

Remark.

This law can take different meanings based on the structure we consider:

- **Entire system**: $m_x = \lambda m_S$
- **Only waiting queue**: $m_q = \lambda m_w$

- **Only server facility**: $m_z = \lambda m_y$

M/M/1

stability: $\rho = \lambda/\mu < 1$

$$p_x(k) = \rho^k(1-\rho), k = 0, 1, \dots$$

$$p_z(0) = 1-\rho, \qquad p_z(1) = \rho$$

$$p_q(j) = \begin{cases} 1-\rho^2, & j = 0 \\ \rho^{j+1}(1-\rho), & j = 1, 2, \dots \end{cases}$$

$$p_s(a) = (\mu-\lambda)e^{-(\mu-\lambda)a}1_0(a)$$

$$p_w(a) = (1-\rho)\delta(a) + \rho\mu(1-\rho)e^{-\mu(1-\rho)a}1_0(a)$$

$$\mathrm{m}_x = \frac{\rho}{1-\rho}$$

$$\mathrm{m}_z = \rho$$

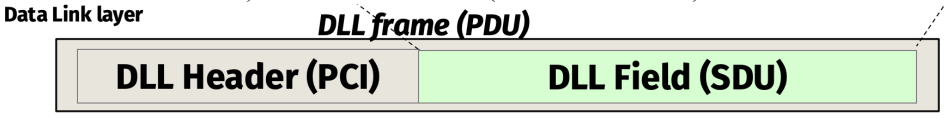
$$\mathrm{m}_q = \frac{\rho^2}{1-\rho}$$

$$\mathrm{m}_s = \frac{1}{\mu-\lambda}$$

$$\mathrm{m}_w = \frac{\lambda}{\mu(\mu-\lambda)}$$

M/M/1 Queue

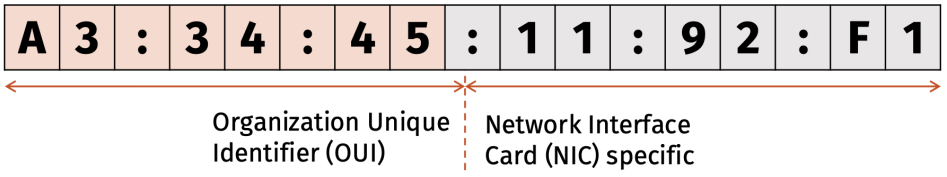
It is important to notice the relationship between layer 2 (DLL) and 3 (Network). The PDU (frame) is made of the header (Protocol Control Information) and the information (Service Data Unit)



Representation

2.3) Address Resolution Protocol (ARP)

How do we identify devices on a PHY level? MAC address:

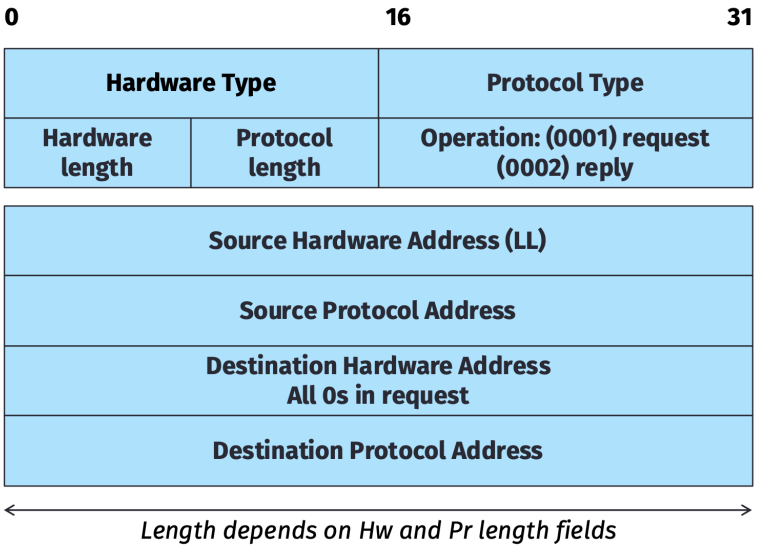


48-Bit

The possible combinations are ≈ 280 trillions which makes it possible to guarantee a 1:1 relationship

When passing packet through a network the ip guides through the whole routing but the jumps from one node to another are directed by the DLL which works only in local (direct) communications. By supposing that all IPs of the route are known the ARP can associate each IP to a MAC and correctly direct the DLL communication on each hop. Therefore ARP is defined at level 2 but is an auxiliary protocol at level 3.

ARP Packet Format



ARP Packet

More details on the header:

- **HW Type:** identifies technology
- **Protocol Type:** higher layer that use ARP
- **HLEN:** DLL address length (MAC = 12 hex)
- **PLEN:** NET address length (IPv4 = 32 bit)
- **ARP OP:** type of arp message: 0001→ request, 0002→ reply

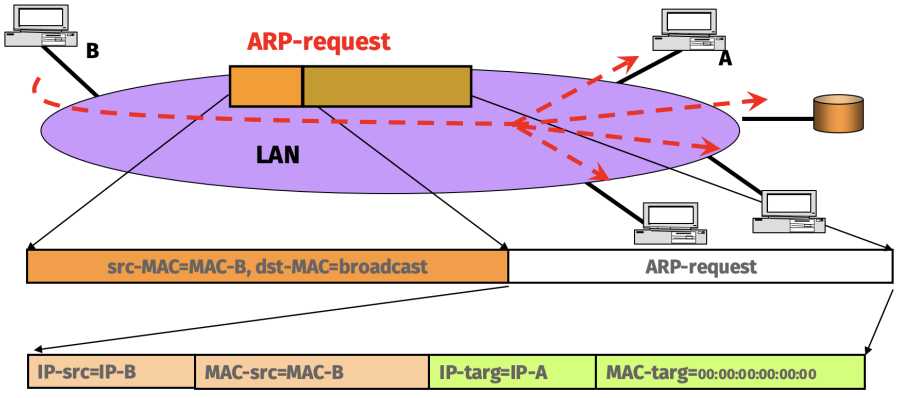
ARP Operation

It is possible to send a **broadcast** packet using the mac address (FF:FF:FF:FF:FF:FF)
It will use this to build a table of mac-ip relations.

How it works:

- Source sends broadcast ARP request where the IP of the target is specified:
  - The header has: scr: MAC B, dst: broadcast
  - The ARP packet has: scr IP: IP B, src: MAC B : target IP: IP A, target MAC: 0
- Receiver sends ARP reply:
  - header has: scr: MAC A, dst: MAC B
  - ARP packet has: scr IP: IP A, src: MAC A : target IP: IP B. target: MAC B

This get's saved in the MAC-IP **forwarding table**

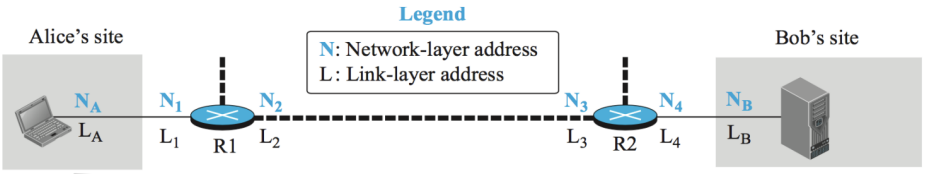


ARP Request

In case of a split network the proxy intercepts and forwards the ARP request and returns the answer wit it's mac address to then manually handle future transmissions.

Remember that ARP cannot leave the subnetwork of a sender!

Example:



Example

Suppose Alice wants to send to Bob.
Alice POV:

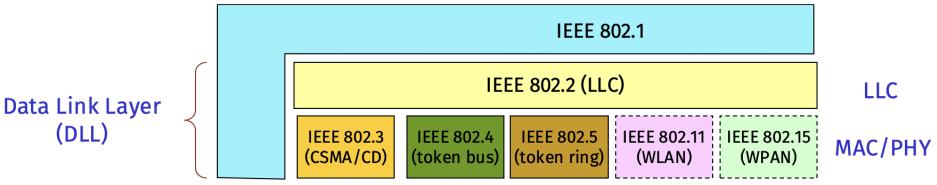
- Network layer forms datagram (NA, NB, Data)
- Network layer finds where to send (N1 in order to reach NB)
- Network Layer must find MAC of R1, if it is unknown it sends ARP
- encapsulation with L1, LA and sends data

R1:

- Arrives data, frames and passes to DL which decapsulates datagram and sends to Network Layer
- NB is known, check routing table and find that next node is N3
- IP of R1 is same of R2 and uses ARP to find MAC of R2
- encapsulation with L3, L2 and sends data

2.4) IEEE 802.x

IEEE 802 is the family of protocols that governs the DLL. The name stands for Institute for Electrical and Electronics Engineers. ISO is the International Organization for Standardization that has identified every IEEE standard



IEEE Family

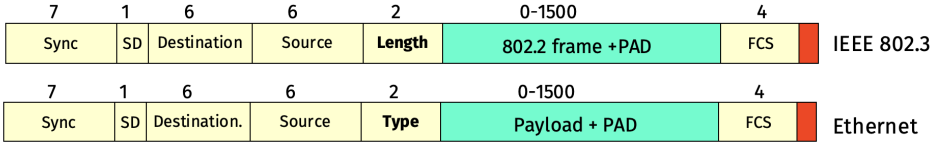
2.4.1) Wired LAN (IEEE 802.3)

The ethernet standard has the following specifications:

- connection less: frame sent when ready
- no flow control: excess frames are dropped, loss recovery managed by upper layers
- no ACK
- RETX in case of collision: CSMA/CD

The commercial ethernet uses a slight variation of IEEE 802.3.

IEEE 802.3 Frame



IEEE vs Ethernet

- **Sync/Preamble:** 56 bits for bit sync (added at PHY)
- **Start frame Delimiter:** framing flag sequence added at phy
- **Destination/source:** 48 bits mac address of destination/source
- **Length:** length of data that is sent with theoretical max of 32kbytes
- **Data:** PDU+PAD for min length (64 bytes) and max of 1.5kbytes
- **Cyclic Redundancy Check** error detection

> **Remark** (Data to remember)**.**
>
> **18 bytes are for header and trailer** $\implies$ **Payload** $\in [46, 1500]$ **bytes**

We define a **max frame length** since we don't want to monopolize the bus.
The **min length** is given by how CSMA works the time to send a bucket of size S with bitrate R is $T_s = S/R$ but for CSMA it must last more than $2T_p$ with T_p the max propagation delay

The backoff has a slot time $T_{slot} = 51,2\mu s$ and the timer is chosen $\in [0, 2^k - 1]$ with $k = \min(n, 10)$. After 16 retx the process is aborted

There are 4 generations: Standard (10mbps), Fast (100mbps), Gigabit (1gbps), 10-Gigabit (10gbps)

The names are chosen in the following way:

$$\text{10Base- } X$$

with:

- 10 the speed in Mbps (can also be other number)
- Base stands for Baseband signal
- X is the type of cable: F fiber, T twisted pair, 2 200m, 5 500m cable
  Example: 1000Base-5 is a 500m gigabit cable

2.4.2) Wireless Lan (IEEE 802.11)

This is the usual WiFi implementation
A Basic Service Set (BSS) is needed

- Access Point as central base station and users connect with authentication
- A network with AP is made ad hoc and cannot send data to other BSS

Extended BSS (EBSS) are 2 + BSS connected via a distribution system. The stations connected with each other don't require AP

IEEE 802.11 automatically adapts the transmission rate, but higher rate require higher SNR

We use CSMA/CA with an initial handshake (RTS) requirement

If handshake (RTS) has a collision no CTS has been sent. If it is not received a collision is assumed $\rightarrow$ backoff

3) Network Layer

The network layer is responsible for host-to-host routing and delivery of datagrams. Layer 3 protocols are implemented in every router. It does the following:

- Framing
- Routing: determine route
- Forwarding: move packets

3.1) Addressing

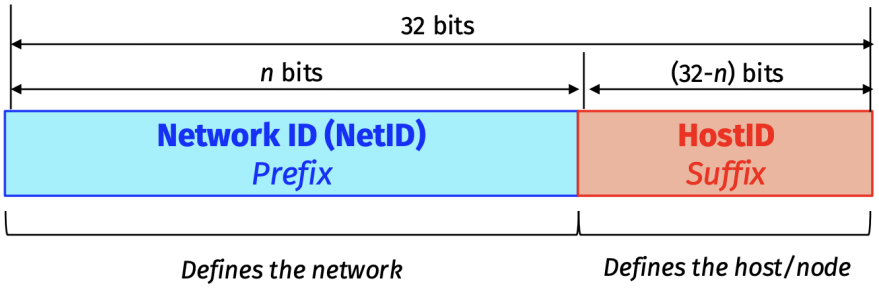
The network layer must **inter**connect **net**works. To do so every network must have an identifier

3.1.1) IPv4

Ipv4 is a **32-bit (2^{4+1}) address** (with a total of 2^{32} addresses) that addresses an unique connection to the internet. It is universal since it is accepted by any host

IPv4 is **hierarchical** as it is divided into two parts:

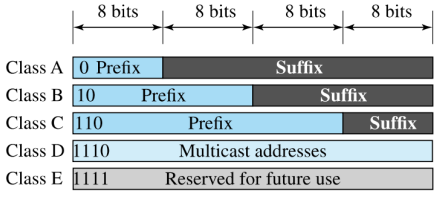
- NetID: prefix of length n and defines the network
- HostID: suffix of length $32 - n$ and defines the host in the network



IPv4

> Nodes on the same network have same NetID and different HostID. Nodes in different networks have different NetID and can have same HostID.

Originally IP were thought as **classfull**, that is there were 3 fixed lengths (8, 16, 24) with 5 spaces (A, B, C, D, E) that had a fixed prefix



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

Classfull

This approach has many flaws, therefore we resort to **classless** addressing, where n has a variable length.

Class	First bits	# net bits	# node bits	# networks	# nodes	from	to
A	0	7+1	$32 - (7 + 1) = 24$	$2^7 - 2$	$2^{24} - 2$	0.0.0.0	127.255.255.255
B	10	14+2	$32 - (14 + 2) = 16$	2^14	$2^{16} - 2$	128.0.0.0	191.255.255.255
C	110	21+3	$32 - (21 + 3) = 8$	2^21	$2^8 - 2$	192.0.0.0	223.255.255.255

Where the -2 in the # nodes tab is for the broadcast/network addresses and the -2 in the A class # networks are the two reserved network addresses

Remark (How to obtain addresses)**.**

The start/finish addresses are done by:

- start: all bits (besides class) to 0
- finish: all bits (besides class) to 1

Network address:

- All HostID bits to 0

Broadcast address:

- All HostID bits to 1

Classless InterDomain Routing (CIDR) & Netmask

Classless addressing sets an arbitrary value for n, therefore we need to define a new way to identify a network.

Definition 5 (Classless InterDomain Routing)**.**

The slash notation represent the **Classless InterDomain Routing** (CIDR) representation:

$$1x8.1x8.1x8.1x8/n,\ n \in [0, 32] \in \mathbb{N}$$

	Operation
Number of Addresses (N)	$N = 2^{32-n}$
Number of Hosts	$N - 2 = 2^{32-n} - 2$
Network Address	last $32 - n$ bits $= 0 \iff$ HostID bits to 0
Broadcast Address	first $32 - n$ bits $= 1 \iff$ HostID bits to 1

Definition 6 (Netmask)**.**

Given an address and a value for n we define the **netmask** as a 32-bit sequence starting with n ones and the rest are all zeroes
Example:

$$192.168.1.7/27 \rightarrow \text{netmask: } 1x8.1x8.1x8.11100000$$

This allows to find the related info with bit-wise operations (NOT, AND, OR)

	Operation
Number of Addresses	**NOT**(mask)+1
Number of Hosts	**NOT**(mask)-1
Network Address	Any address in block **AND** mask

	Operation
Broadcast Address	Any address in block **OR NOT**(mask)

Example.

Suppose we have the ip:

$$192.168.1.7/27 \iff 11000000.10101000.00000001.00000111/27$$

CIDR:

- *Number of Addresses* = $2^{32-27} = 32$
- *Network (first) Address:*

$$32 - 27 = 5 \to 11000000.10101000.00000001.00000000/27 \to 192.168.1.0/27$$

- *Broadcast (last) Address:*

$$32 - 27 = 5 \to 11000000.10101000.00000001.00011111/27 \to 192.168.1.31/27$$

- *From here we can see that we have exactly 32 addresses (from .0 to .31)*

Netmask:

- *Netmask:*

$$11111111.11111111.11111111.11100000 \to 255.255.255.112$$

- *Number of Addresses:*

$$00000000.0000000.00000000.0001111 + 1 = 10000 \overset{\text{dec}}{=} 32$$

- *Network Address:*

$$\begin{array}{rcccc} & 11111111 & 11111111 & 11111111 & 11100000 \\ & 11000000 & 10101000 & 00000001 & 00000111 \\ \text{AND:} & 11000000 & 10101000 & 00000001 & 00000000 \end{array} \to 192.168.1.0/27$$

- *Broadcast Address: (OR)*

$$\begin{array}{rcccc} & 00000000 & 00000000 & 00000000 & 00011111 \\ & 11000000 & 10101000 & 00000001 & 00000111 \\ \text{OR:} & 11000000 & 10101000 & 00000001 & 00011111 \end{array} \to 192.168.1.31/27$$

- *This is the same as before*

Here is a quick reference table with some useful numbers

Binary	Decimal
11111111	255
11111110	254
11111100	252
11111000	248

Binary	Decimal
11110000	240
11100000	224
11000000	192
10000000	128

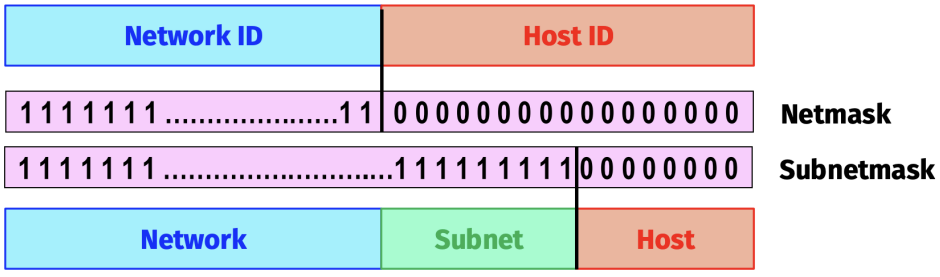
The association that assigns blocks of addresses to an ISP is called **Internet Corporation for Assigned Names and Numbers (ICANN)** and has to follow these rules:

- The requested number of addresses (M) must be a power of 2
- The prefix length of each block (n) is then $n = 32 - \log_2 M$
- The network address of the block will have $\log_2 M$ zeroes

> **Remark.**
>
> If we need k addresses we need to ask for $M = 2^{\lceil \log_2 k \rceil} \implies n = 32 - \lceil \log_2 k \rceil$

Subnetting

A large network can be divided into further subnetworks by creating a subnetmask



Example

Here the initial HostID is divided into SubID + (new) HostID. From here the same rules as before work. Let's call n the netmask, $m \leq 32 - n$ the subnetmask, then:

- HostID has length $32 - n - m$
- Number of subnetworks: 2^m
- Number of Addresses per subnetwork: 2^{32-m-n}

In real world scenarios both a class and a netmask are assigned. For subnetting the netmask > class NetID bits. The class NetID bits can't be changed and thus **the number of bits m for the subnet is given by netmask-class HostID bits**.

Subnetting is achieved by having a netmask different from the bits of the class NetID

The following steps are used fro a correct subnetting:

- N_{sub} must be a power of 2
- each network has a prefix of $n = 32 - \log_2 N_{sub}$
- The starting address in subnetwork should be divisible by the number of addresses in the subnetwork → we need to start the process by assigning first addresses to larger subnetworks. ???

here are some useful examples:

Example.

Suppose to have a class B (n=16) address with netmask of $255.255.255.0 = 24$. *Then we have:*

- $32 - 24 = 8$ *HostID bits*
- $24 - 16 = 8$ *SubnetID bits* $\implies 2^8 = 256$ *addresses*

Suppose to have the following IP: $147.162.8.3/24$ *we can find the SubID:*

- *Rewrite the IP:* $147.162.8.3 \to 10010011.10100010.00001000.00000011$
- *We know that the first 16 bits are used for the NetID, and the following 8 are the SubID*
- *The SubID is* $00001000 \overset{\text{dec}}{=} 8$ *and the HostID is* $00000011 \overset{\text{dec}}{=} 3$

Example.

Create subnets with at least 500 hosts with network address $132.78.0.0$ *and NetID of 16 bits:*

- *Since 500 is not a power of 2 we find the closest bigger power* $\to \log_2 500 = 9 \to 2^9 = 512$
- *Then we have HostID=9 and thus* $n = \text{NetID} + m = 32 - \text{HostID} = 23 \to m = 7$
- *We subnetted to have 128 subnets with 510 hosts each*

Example.

Create at least 1000 subnets with Network address $128.234.0.0$ *and NetID 16:*

- *1000 is not a power of 2* $\to m = \log_2 1000 = 10 \to 1024$ *subnets*
- *Then we have that* $n = \text{NetID} + m = 32 - \text{HostID} = 26 \to \text{HostID} = 6$
- *We have created 1024 subnets with 62 hosts each*

Example (Uneven Subnetting)**.**

We have the addresses $14.27.74.0/24$ *and want to split them in 3 subnetworks of type:*

1. *one subblock of* $10 \to 16$ *addresses* $\implies \text{HostID}_1 = 4$
2. *one of* $60 \to 64 \implies \text{HostID}_2 = 6$
3. *one of* $120 \to 128 \implies \text{HostID}_3 = 7$

Since we already calculated the HostID in the 3 cases we already know that $\text{HostID}_{\max} 32 - 24 = 8$ *satisfies our requirements. We use rule 3* $\to$ *we start with the biggest subnetwork:*

- $n_3 = 32 - \text{HostID}_3 = 25$ *and thus the network address is* $14.27.74.0/25$ *and broadcast* $14.27.74.0.01111111/25 \to 14.27.74.127/25$ *(128-1 difference)*
- $n_2 = 26$. *Notice that we can't use net address of* $14.27.74.0/26$ *since these addresses are already taken by subnet 3. All addresses until* $14.27.74.128$ *are taken, these relate to SubID* $00, 01$ *so we choose*

SubID=10 *and we have net/broad addresses* $14.27.74.128/26$ *and* $14.27.74.191/26$ *(64-1 difference)*
- $n_1 = 28$. *As before the SubIDs starting with 0 or 10 are already taken, so we use* 1100. *We have net/broad addresses* $14.27.74.192/28$ *and* $14.27.74.207/28$ *(16-1 difference)*

From the last exercise we can derive the following important remark:

Remark (No overlapping)**.**

Subnets can't have blocks that overlap. Since the IP datagram doesn't contain the netmask it will cause confusion to the routing.

Moreover when deciding the network address of a block it must start with an address with HostID=0. If all the addresses from the network to the broadcast address (HostID=1) aren' already occupied the block can be used for subnetting

From here another remark can be made:

Remark 7 (Constraints on Network and Broadcast Addresses)**.**

The final conclusion is that a network address must have HostID=0 and the broadcast address must have HostID=1. If this is not the case the given address is not a network/broadcast address or the network is badly set up.

Supernetting

Class A, B networks are running out, therefore we can group many class C networks to join them into a bigger network

Example.

We have the following blocks

$$193.23.136.0/24$$
$$193.23.137.0/24$$
$$193.23.138.0/24$$
$$193.23.139.0/24$$

All these go from .0 *to* .255 *so the full range from* $193.23.136.0/24$ *to* $192.23.139.255/24$ *are covered. These are* $256 \cdot 4 = 2^{10} = 1024$ *addresses, exactly the number of addresses of a /22 network. From here it is clear that these can be grouped into the single block*

$$193.23.136.0/22$$

3.1.2) IP

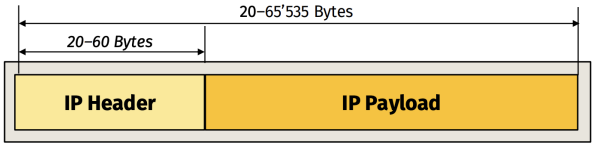
IPv4 is

- **Connectionless** protocol that uses the datagram approach. It is responsible for packetizing, forwarding and delivery of packet. It is said to be
- **Best-Effort/Unreliable:** packets can be lost corrupted arrive OOO. It must be paired with a reliable transport-layer protocol

IP MTU:
Based on the protocol each router decapsulates the IP datagram and encapuslates it in another frame. Larger datagrams are therefore fragmented to fit in the MTU. The header is usually straight up copied and never modified during the fragmentation process, only **flags, offset and length change**.

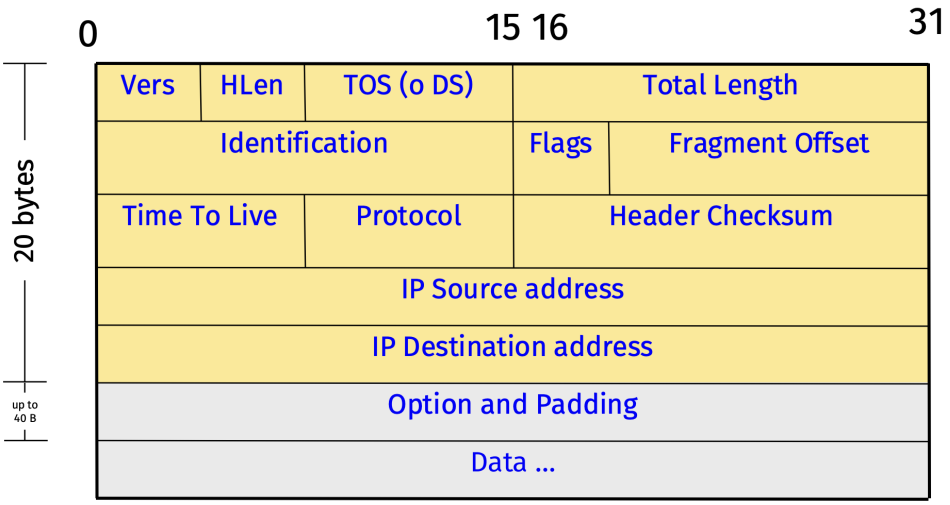
Datagram

Here is the datagram of the IP in detail:



IP Datagram

Let's focus on the header:



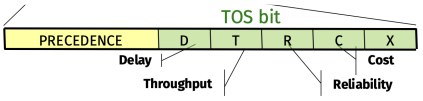
IP Datagram Header

Name	Length (Bits)	Description	More in depth
Vers	4	version of protocol used	clearly always 0100
HLeng	4	stores the total length of the header in *bytes*	The minimum is defined as $0101\|\|_2 = 5\|\|_{10} \rightarrow 5 \cdot 4 = 20$ Bytes. The maximum is $1111\|\|_2 = 15\|\|_{10} \rightarrow 15 \cdot 4 = 60$ Bytes
Type Of Service (TOS)	8	defines how datagram should be handled.	See below
Total Length	8	length of header + data in *bytes*	Length of data can be found by data=Total Length - HLeng
Time To Live (TTL)	8	maximum # of hops before getting dropped.	On drop an ICMP may be sent
Protocol	8	Upper layer protocol used	TCP = 06, UDP=17
Checksum	16	rough form of error detection	covers only header
IP Source/Destination	32x2	IP Address of Source/Destination	These can't be changed while in flight. Netmasks are NOT included. Routing tables should handle this

Name	Length (Bits)	Description	More in depth
Identification	16	helps destination in reassembling datagram (all fragments with same flag should be assembled into one datagram)	Same for all fragments
Flags	3	Fragmentation control	R = reserved. D = don't fragmen (if bigger than MTU send error message). M = More Fragment, this is not the last fragment
Offset	13	Relative position of fragment with respect to the whole datagram	It is the offset of the data in the og datagram measured in units of 8 bytes (first bit/8)
Options/padding	up to 40	used for testing and debugging	

Remark (TOS in detail)**.**

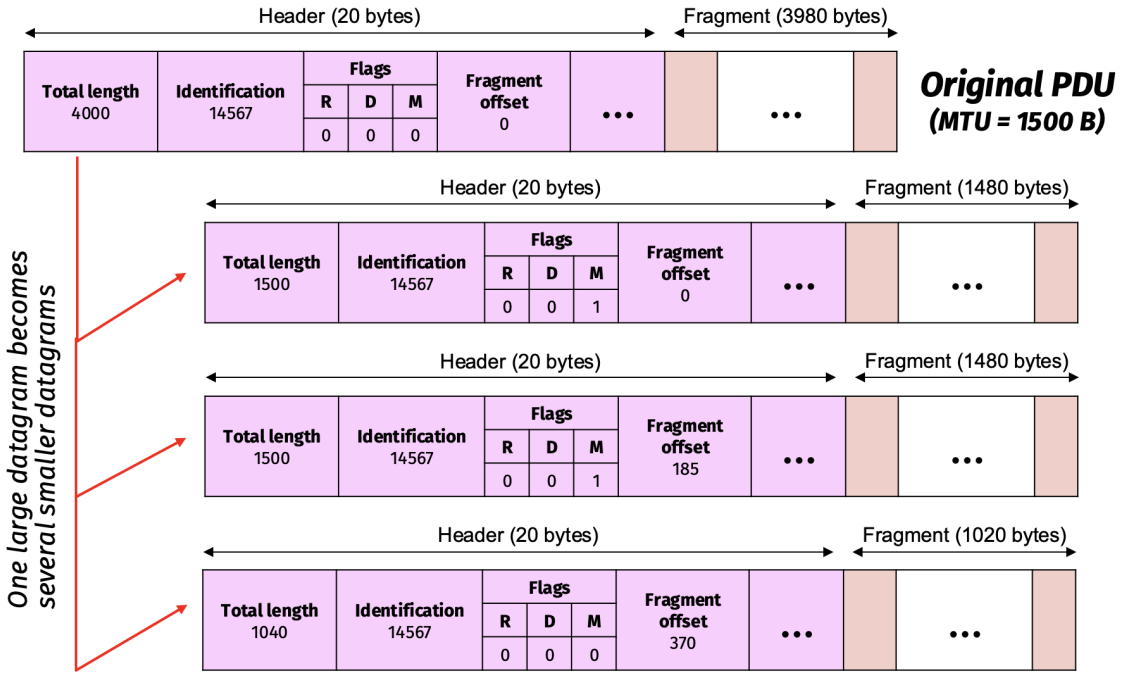
Here are some more details on the TOS



TOS

TOS Bits	*Description*
0000	Normal (default)
0001	Minimize cost
0010	Maximize reliability
0100	Maximize throughput
1000	Minimize delay

TOS Bits



Fragmentation Example

The biggest problem with fragmentation is that the loss of one fragment means to lose the entire datagram. To avoid this the IP layer shouldn't exceed the MTU of the network(s) it will go to

Fragmentation **changes** the following fields:

- **Length, Flags, Offset, Header Checksum**
  And doesn't change:
- Version, TOS, Id, TTL, Protocol, IP src/dst

3.1.3) DHCP

How can we assign IP and netmasks to devices connected do a block of addresses?

A deprecated protocol was **BOOTP** where everything was manually set and remains static → deprecated.

	BOOTP	**DHCP**
IP Assignmente	Only when booting MAC correspondance in look-up table	Any time when OS is loaded Dynamic addressing
IP Lease	System has to restart (permanent IP)	Automatically rebind or renew
Error	Prone to error	Immune since autoreconfigurable
Flexibility	Is only static	Handles mobile connections

What are the advantages of DHCP and mainly **dynamic addressing?**
DHCP laeses out IPs to devices for a limited time. This allows to manage a large number of devices that aren't always connected to the network. By doing so we can manage many more devices than IPs since if host doesn't use the network anymore he loses it's IP.

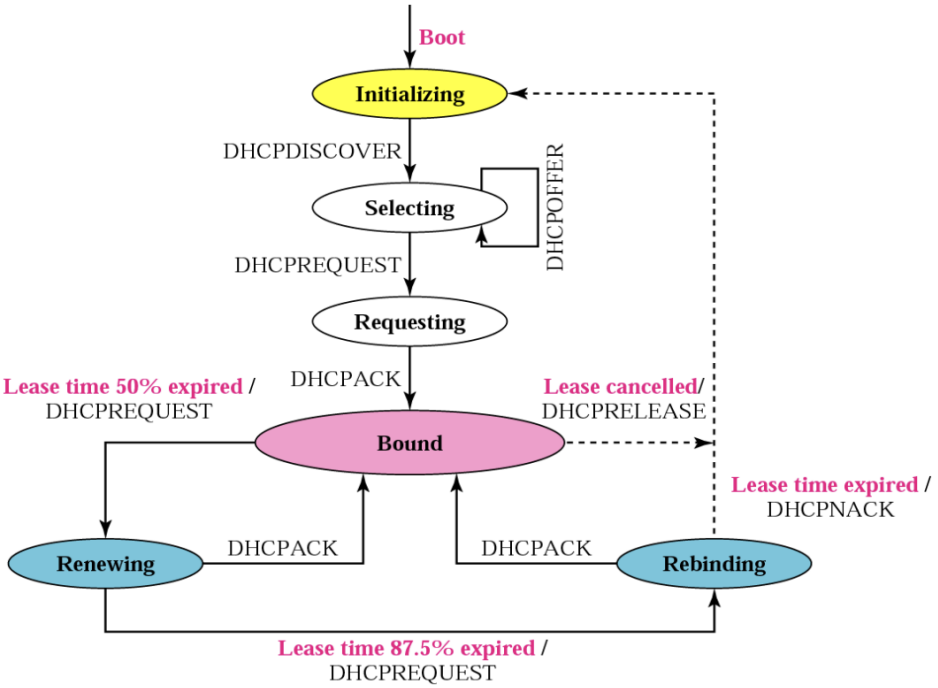
Protocol workings:

1. **Discovery:** a host (client) sends via broadcast a DHCPDISCOVER message containing it's MAC address.
2. **Offer:** Each DHCP server intercepts and replies with a broadcast DHCPOFFER that contains it's ID and proposed IP+lease time
3. **Request:** The host selects one offer and accepts it with a broadcast DHCPREQUEST message that contains the ID of the server that gave the offer.
4. **ACK:** the final broadcast packet DHCPACK confirms the IP and gives a path with all info for configuring the interface via FTP (netmask, default gateway, DNS)
5. **Release:** This is to end the lease from the host. A unicast packet DHCPRELEASE is sent to the server

DHCP works by assigning ip through leases.

1. Host sends DHCPDISCOVER request over broadcast
2. Each DHCP replieas with a DHCPOFFER broadcast containing ip source: dhcp ip, ip destination proposed ip address+lease time
3. Client sends broadcast DHCPEQUEST with its new ip
4. server associates ip to machine replies with broadcast DHCPACK

what is the DHCPACK? It contains all info needed to configure: netmask, gateway, dns



State Diagram

3.1.4) Internet Control Message Protocol (ICMP)

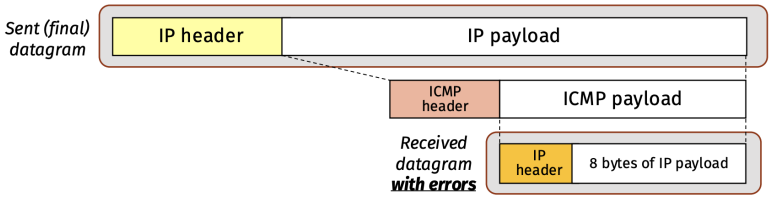
ICMP is a companion to IP to compensate for

- error reporting: reports problems to source
- host and management queries, query messages: get info from other router/host, probes liveliness and RTT

Moreover **no ICMP message is sent for:** multicast/special address, not the first fragment, ICMP message

All messages contain:

- Ip header that caused error
- first 8 bytes of data



Encapsulation

Therefore it is both inside and on top of IP
The ICMP header is **8 bytes**
Therefore the payload has to be IP header + ICMP header smaller than the max MTU, usually that is 28 bytes smaller

ICMP Error-Reporting Messages

This is the datagram used by all types of error messages, but code can have a different range:

type (3)	code (0-15)	Header checksum
Not used (16 bits, all zeros)		Next-Hop MTU (if code=4, otherwise all zeros)
header + first 64 bits of the IP datagram that caused the problem		

- **Type 3: Destination Unreachable**, most common
  The code specifies a reason explained in the following table

Code	Description
0	Network unreachable error.
1	Host unreachable error.
2	Protocol unreachable error (the designated transport protocol is not supported).
3	Port unreachable error (the designated protocol is unable to inform the host of the incoming message).
4	The datagram is too big. Packet fragmentation is required but the 'don't fragment' (DF) flag is on.
5	Source route failed error.
6	Destination network unknown error.
7	Destination host unknown error.
8	Source host isolated error.
9	The destination network is administratively prohibited.
10	The destination host is administratively prohibited.
11	The network is unreachable for Type Of Service.
12	The host is unreachable for Type Of Service.
13	Communication administratively prohibited (administrative filtering prevents packet from being forwarded).
14	Host precedence violation (indicates the requested precedence is not permitted for the combination of host or network and port).
15	Precedence cutoff in effect (precedence of datagram is below the level set by the network administrators).

Code Table

- **Type 11: Time exceeded**
  has two codes:

0. TTL reached 0, returned to source
1. Generated by destination if frames are missing

- **Type 12: Parameter Problem**
  has two codes:

0. Header of IP datagram has a malformed field
1. Option is unknown or cannot be executed

ICMP Query Messages

This uses **echo request (type 8) and echo reply (type 0)** to test liveliness of other nodes. If an echo request is received a echo reply is sent to the og sender. This is used for pinging and RTT calculations

One important ICMP based command is **traceroute:**

- it sends the first message wit TTL=1. The router discards the packet and sends **time exceeded ICMP error** and the ip of the router and logs it
- sends i-th message with TTL=i and logs the ip of reached router
- the last message reaches the destination but at the wrong port: **destination unreachable ICMP error**. It logs the arriving ip and knows to stop

3.1.5) Network Address Translation NAT

An intranet is a private network that uses TCP/IP protocol stack. It can be connected to the internet but external (not useful for internal nodes) traffic never enters it. Only the gateway uses a public IP, and internal nodes are assigned to a private IP that might not be unique outside of the network (but must be unique inside).

Class	IP address range	Number of addresses	Largest subnet mask	HostID size
A	10.0.0.0 – 10.255.255.255	16'777'216	255.0.0.0 (/8)	24 bits
B	172.16.0.0 – 172.31.255.255	1'048'576	255.240.0.0 (/12)	12 bits
C	192.168.0.0 – 192.168.255.255	65'536	255.255.0.0 (/16)	16 bits

Private IP Ranges

This allows me to connect more devices than the maximum allowed by the ICANN

Packets that are directed to private Ip addresses are dropped by routers. We need a proxy, but this would require a proxy for every application

NAT can map private to public addresses by having a pool of public addresses that dynamically maps to a private address in the **NAT table** and acts as a "bigger" proxy to allow bi directional communications.
This table is dynamic: the host and the NAT create a session where the private and public addresses are binded together. Once the session is over the binding is broken and the public IP is free to be used

> **Remark.**
>
> ICMP messages carry IP addresses in their body. These must also be changed

	Basic NAT	**NAPT**	**Twice NAT**
Session Initiator	Outbound	Outbound	Both directions
Number of Connections	= NAT IP pool	$\geq$ NAT IP pool	$\geq$ NAT IP pool

Basic (outbound) NAT

This NAT can only start connections from the intranet to the internet (outbound). Moreover If the pool of the NAT is saturated some sessions cannot start

Traditional NAT - Network Address Port Translation (NAPT)

This nat associates a [Private IP, Source Port] with a [Public IP, NAT source Port].
This allows to reuse the same public IP for more hosts, if the port number is different. The NAT source port is randomly chosen between the NAT-enabled gateway

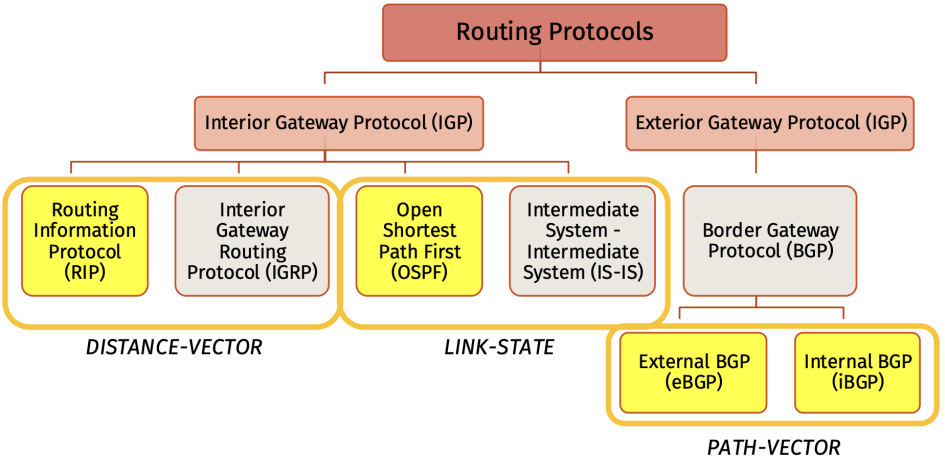
Bidirectional NAT - Twice NAT

Supports both internal and external session initiation. But how? With the use of a **DNS** (Application Layer)

3.2) Routing

Routing is the process that makes it possible to bring a packet from source to destination.

We can see the internet as a weighted graph and the best route is the **least cost route (LCR)** ad can be found based on different algorithms, these are applied by different routing protocols.



What are our cost metrics? The cost between two nodes $c_{i,j}$ can have different values:

- **Min number of hops:** $c_{i,j} = 1$
- **Min e2e delay:** $c_{i,j} = L/R + t_p$ where L is the packet size, R the bitrate and t_p the propagation time
- **Min e2e packet loss rate:** $c_{i,j} = -\log[P_{ok}]$ where P_{ok} is the success probability over the link
- **Max long-term throughput:** $c_{i,j} = 1/R^n$ this cannot be used with Djikstra

3.2.1) Algorithms

What does "best" mean? It is the path that minimizes e2e delay and packet loss with the minimum # of hops and max throughput. These are represented as weights in the algorithms. All these algorithms produce a **routing table** that stores three columns:

- address of destination network
- address of next router
- cost to reach the destination

Distance-Vector	Link-State
Routing Information Protocol (RIP)	Open Shortest Path First (OPFS)
Intra Autonomous Systems	Intra Autonomous Systems
Algorithm: **Bellman-Ford**	Algorithm: **Dijkstra**
Forwarding of (a copy of) the whole table (Distance Vector)	Forwarding of just the link updates
It does not require the network topology	It requires the network topology
Frequent updates: **slow** convergence	Event-based updates: **fast** convergence

Comparison

Distance-Vector: Bellman-Ford's Algorithm (RIP, IGRP)

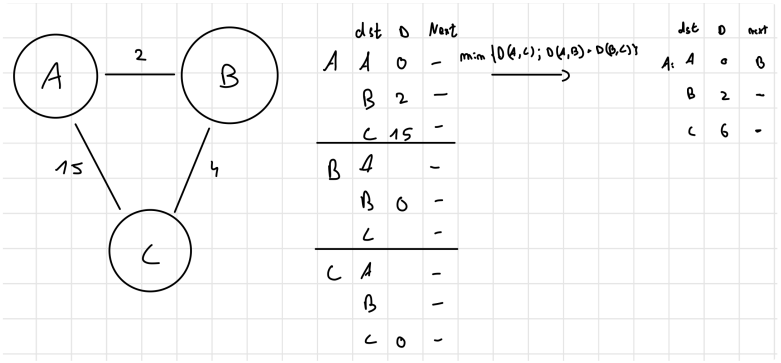
This approach **doesn't need a global knowledge of the network,** it dynamically builds the graph.
Each node contains a table with the all nodes as the destination, the distance from these nodes and the next node to go to to reach them. Each node builds this table based on it's neighboring tables. These get calculated every approx 30 seconds and at convergence will have the most efficient paths. This is said to be **decentralized.**

Each node identifies the neighbors with HELLO packets and computes the delay with ECHO. Then it creates a Distance Vector (DV) with this information:

- list of neighbors
- cost of link
- list of nodes that represent "next hop"

Each 25-35 seconds or on DV changes each node sends it's DV to the neighbors.

- Initially the DV will only have the distance of the direct neighbors and no next hop.
- When comparing the DV with the neighbor DV this slowly expands since we now know what next hop+distance lets us achieve the connection to a far away node.
  - The next hop node is based on the distance: the next hop node is chosen based on which node will allow to packet to go the lowest distance for the desired destination



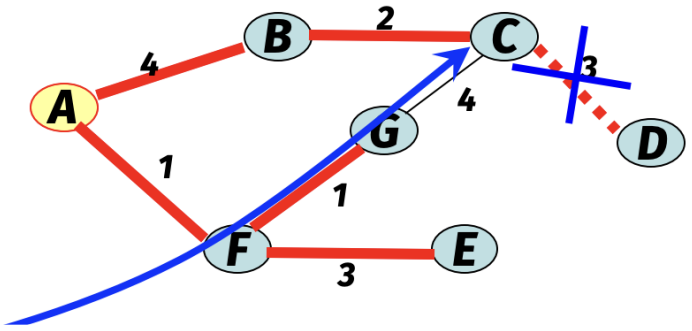
Simple Example

What happens in case of **failure?**
Suppose a node stops receiving HELLo packets from a neighbor. When a TO expires the node will update the DV by removing all entries that have the failed node/link as next hop. It then recalculates the distances (possible only with neighbors) and sends the updated version forward to the affected nodes.

It is possible that a failed link divides the graph into **two spanning subgraphs**
Now at every update the path for a node in the other subgraph will increase by 1 and start to **count to infinity**, we can solve this issue by:

- **limit infinity:** if distance is more than 16 it is considered infinite (but this works only if the max distance is always < 16)
- **Hold down:** when a router is informed that the dst is not reachable it ignores any DV update for 60s in order to prevent obsolete information to propagate
- **Trigger Update:** topology changes are differentiated from other notifications
- **Split horizon**

Split horizon is a bit more complex:
If a path to a node fails the DV of each node are not sent to the neighbors that are in the path for the failed node. and the route to the failed node is eliminated



Example

In this case From B to D we go trough C. B doesn't send the DV to C.

Actually if we delete the path to the failed node the neighbors can't guess if this is due to split horizon or if the node really doesn't know the path, so we use **poison revenge**, that is we set the distance to the failed node as +infty

Link-State: Dijkstra's Algorithm (OSPF, IS-IS)

To apply this algorithm we need to have **global knowledge** about the network. Each node needs to know it's neighbors (done through HELLO packets), and its cost (ECHO). It creates a Link State Packet (LSP) with this information to send to the router that generates the graph. Therefore it is **centralized.**

```
1 function Dijkstra(Graph, source):
2 create vertex set V
3 for each vertex v in Graph {              // initialization
4         dist[v] ← INFINITY                // unknown distance from source to v
5         prev[v] ← UNDEFINED               // previous node in optimal path from source
6         add v to Q }                      // all nodes initially in Q (unvisited nodes)
7 dist[source] ← 0                          // distance from source to source
8 while Q is not empty {
9         u ← vertex in Q with min dist[u]          // node with the least distance
10        remove u from Q                           // will be picked first
11        for each neighbor v of u {                // where v is still in Q
12                alt ← dist[u] + length(u, v)      // cost of alternative path
13                if alt < dist[v] {                // a shorter path to v has been found
14                        dist[v] ← alt             // update cost-estimate of node v
15                        prev[v] ← u  }            // update predecessor list of node v
16        } // end for
17 } // end while
18 return dist[], prev[]
```

Pseudocode

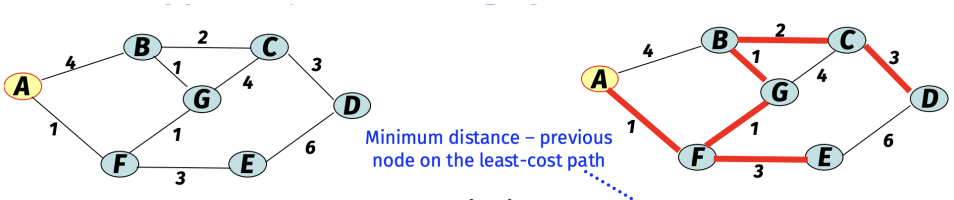
Essentially we spread out from the starting node (n) by checking all neighbors.

1. **Initialization:** In the first check of all neighbors we can surely tell that the node v with the min distance is the shortest distance $n - v$ possible and it is removed from PQ (the distance is now permanent).
2. We check all the nighbors of v. We set the distanve $\min(\text{old distance}, \text{v.dist}+\text{dist}(\text{v,u}))$. By the same logic of step 1) this time the shortest distance will be removed from the PQ. (If a shorter distance were possible, the PQ wouldn't have chosen node v as the shortest distance in the step before since the minimum distance is 1 for each link). Repeat step 2 on the node u. until we reach completion

In a **graphical way** we can see it working in the following way:

1. start from root and set it to (0,p). Then update all neighbors with (d,t). Go to the nearest node and update it with (d,p).
2. Update all neighbors of this node with $d = \min(\text{old d}, \text{this node d} + \text{link d})$.
3. Go to lowest distance node and set it to (d,p). Then do step 2) and repeat until all nodes are permanent

During the operation we set the values (distance, permanent/tmp) and prev. node. The first couple is useful for the algorrithm itself, the last value is set to find the final path since we hop from one to the next. In the case with equal distance from more nodes we choose the one that brings to the least amount of hops



Destination nodes

Step	visited	A	B	C	D	E	F	G
0	{A}	-	4-A	X	X	X	1-A	X
1	{A,F}	-	4-A	X	X	4-F	1-A	2-F
2	{A,F,G}	-	3-G	6-G	X	4-F	1-A	2-F
3	{A,F,G,B}	-	3-G	5-B	X	4-F	1-A	2-F
4		-	3-G	5-B	8-C	4-F	1-A	2-F
5	{A,F,G, B,C,D}	-	3-G	5-B	8-C	4-F	1-A	2-F

Dijkstra Example

Path Vector: Spanning Trees (eBGP, iBGP)

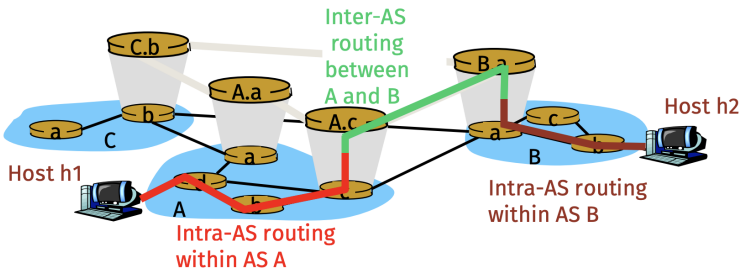
This algorithm allows to apply specific proprietary conditions for routing. This uses the **spanning tree** algorithm that is similar to Bellman-Ford but the distance is a custom path function

3.2.2) Routing protocols

Each ISP is seen as an **autonomous system (AS)** and two different **Gateway Protocols** are used

- Interior IGP: inside of AS, can be different for any AS
- Exterior EGP: outside of AS. Here each **Border Gateway (BG)** of any AS notifies other **Interior Gateways (IG)** about the addresses that are reachable trough such AS

Each AS has a unique ID (32 bit) given by the **Regional Internet Entries (RIE)**



Example

Performance	RIP	OSPF	BGP
Traffic	Local update messages only to neighbors (do not create traffic)	Complex message format (can create heavy traffic and use a lot of bandwidth)	Exchange a lot of messages (create heavy traffic and use a lot of bandwidth)
Convergence	Slow convergence and loop and count-to-infinity	Fast convergence (but only once Dijkstra is completed)	Free from loops and count-to-infinity
Robustness	NOT robust (if there is a failure or corruption in one router, the problem will be propagated to all routers and the forwarding in each router will be affected)	ROBUST (each router is independent and does not depend on other routers in the area)	NOT robust (if there is a failure or corruption in one router, the problem will be propagated to all routers and the forwarding in each router will be affected)

Comparison

Interior Gateway Protocol (IGP)

The first we study is **Routing Information Protocol (RIP)**. This uses:

- Bellman-Ford
- Number of hops
- 15 max hops (16 is infty)
- DV every 25-35 seconds
- RIP packets are UDP datagrams

The **v2** adds

- connectivity info (route tag+next hop)
- auth
- classless routing
- multicasting to 224.0.0.9 (all RIP)

And there are 3 timers:

- Periodic timer: random every 25-35 seconds to send DV so that not all share at same time
- Expiration Timer: 180s. After that hop count is set to infty=16
- Garbage Collector: 120s after expiration the entry is purged

For **larger AS** we use **Open Shortest Path First (OSPF)**. This uses:

- Link state with advertisements
- Custom cost metrics
- message in IP packets

OSPF introduces the idea of "area", a collection of linked networks that are summarized by **Area Border Routers**. This is done to prevent flooding of LS information. All the areas must be connected to **area 0 that is a backbone**

There are multiple kinds of links:

Type	Generated by	Flooded within	Purpose
Type 1 (Router LSA)	All OSPF routers	Same area	Describes router links
Type 2 (Network LSA)	DR on multi-access networks	Same area	Lists routers on the network
Type 3 (Summary LSA)	ABR	Other areas	Advertises inter-area networks
Type 4 (ASBR Summary LSA)	ABR	Other areas	Advertises location of ASBR
Type 5 (External LSA)	ASBR	Entire OSPF domain	Advertises external routes

Table of links

1. this has transient links (link to router that allows transient traffic), stub links (no transient traffic), p2p link (link to another router in same area) and sends IP of router
2. The message carries IP of **Designated Router (DR)**, that is the router that advertises the network and all IP of routers (+netmask)
3. sent by ABR to advertise presence of another area. It does so by converting Type 1 to type 3. It glues the areas together
4. advetises **Autonomous System Border Router (ASBR)**, that is a router connected to other AS the IP of ABSR is sent
5. Sent by ASBR to advertise exsistence of network outside of AS and sends network ip

Exterior Gateway Protocol (EGP) actually BGP

We study just **Border Gateway protocol (BGP) v4**

- Based on path-vector
- provides info on reachability of other networks
- each ASBR has an eBGP to obtain info from other neighbor AS
- all routers have iBGP that uses ASBR to propagate reachability info

External BGP

Two border gateways BGP peer and BGP speaker try to create TCP on port 179 and send info on network in areas

Internal BGP

TCP session on port 179 between any 2 pairs of routers

3.2.3) Forwarding

As we studied IP is **connectionless**, but today it is becoming a **connection-oriented** protocol using virtual circuits. The forwarding can be **direct or indirect**

Direct Forwarding

If source and destination have same NetID they are in the same local network.
Then the source checks for MAC of destination
Ip frame get's encapsulated to MAC and forwarded

With netmask direct forwarding is executed when IP(dst) && NM(x) = IP(x) && NM(x)
This means that the IP with the netmask of interface X has the same NetID of the interface X

Indirect Forwarding

The NetID is not the same, then they are not in the same network
The source passes the datagram to a router (default gateway) by finding the MAC of the router
The router forwards it until it is possible to do a direct forwarding

The MAC addresses are always the two nodes communicating but the ip's are the intended source destination of the ip datagram

With the netmask, once direct forwarding is excluded, the routing table is used in one of the two ways:

- longest mask matching
- default gateway

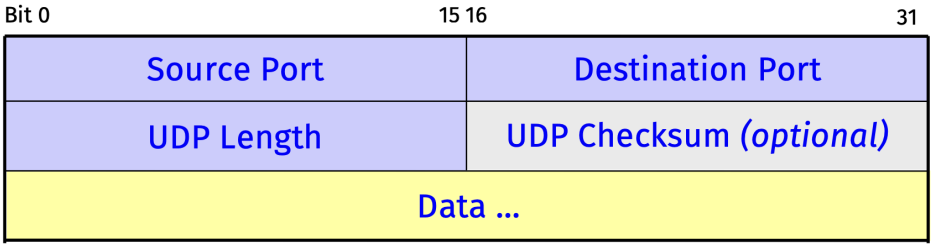
Route Aggregation

When classless addressing is used the routing table drastically increases, therefore **hierarchical addressing** is used.
They are distributed based on advertisements

4) Transport Layer

4.1) User Datagram Protocol (UDP)

This is the baseline protocol to exchange data over networks



Structure

It is **connectionless**:

- all datagrams independent
- no sequence numbers
- no error/flow control and optional checksum
- can arrive OOO

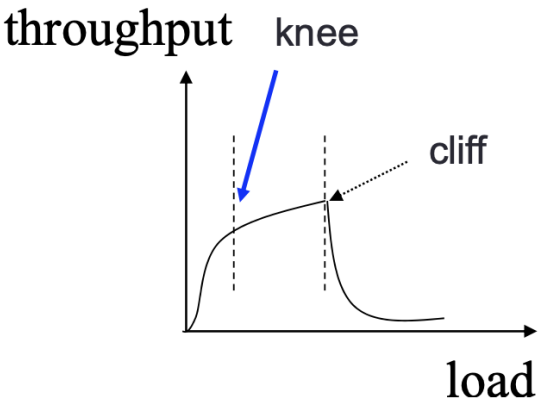
However UDP can be used since it allows to have proprietary error control and minimizes delay.

4.2) Transmission Control Protocol (TCP)

4.2.1) Congestion Control

A network with a slow link can get congested, once a queue is overflown packets are dropped and the network collapses as it only sends RETX. TCP has the scope to provide fair and efficient network allocation to prevent/avoid collapse

We actually study **control and avoidance** which operate in 2 separate moments:
-Congestion Avoidance (CA) at knee
-Congestion Control (CC) at cliff



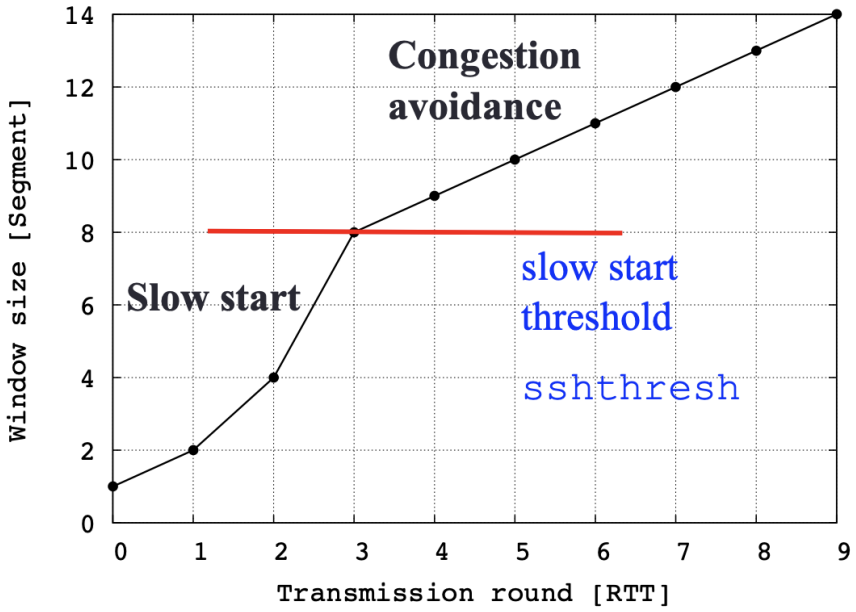
TCP follows these 3 design criteria:

- **Congestion Control & Avoidance:**
  - control: slows down tx rate if capacity drop is exerpienced
  - avoidance: slow down tx rate if an incoming cliff is known
- **Efficiency**: most efficient at knee without excessive delay
- **Fairness:** a fair allocation means that all have equal rates

We also define a **window W of unacknowledged packets** that can be sent in each RTT. However **W** is dynamic and adapts to the network

Probing Principles:

- **SlowStartThreshold** (ssthresh): is a running estimate of the aviable channel capacity
- **Fast Probing** (up to ssthresh): *slow start phase* with exponential increase and $W \to 2W$ at each RTT
- **Slower Probing** after threshold: *congestion avoidance,* linear increase and $W \to W + 1$



Example

Both the sender and receiver can impose flow control:
Sender:

- **Slow Start (SS):** at start/timeout $W \to 2W$
- **Congestion Avoidance (CA):** when full link capacity is reached $W \to W + 1$
  Receiver: rwnd

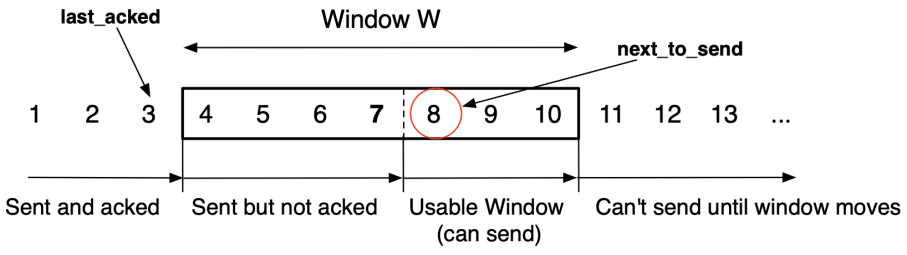
The window size is obtained as

$$W = \min(\text{cwnd}, \text{rwnd})$$

the minimum between the congestion and the receiver window (susually constant. Gets sent with every ACK). CWND changes based on ACKs recieved, RWND is sent along each ACK and can also change. The window is a sliding window

The **new packet can be transferred if**

$$\text{next\_to\_send-last\_acked} \leq W$$



Sliding windows

Terminology:

- Max Sender window size $\mathbf{W_{max}}$ (not specified in example)
- Last ACK Received (**LAR**) is the lower window edge (before) (3 in example)
- Last Frame that can be Sent (**LFS**) is the upper window edge (before) (10 in example)
- **Flightsize** is the number of sent but un ACKed pkts=last_seqno-ack_no+1 (4-7 in example)

If we compare window size to BDP we have

$$W \times t_{tx} = W \times \frac{MSS}{rate} \geq RTT \iff W \times MSS \geq rate \times RTT$$

The **ideal window size** is then: $bitrate \times RTT$

- if W< BDP we are inefficient: wasted bandwidth
- if W > BDP we have queueing: increased RTT and potentially packet loss

Therefore the **throughput** is

$$\text{Throughput} \leq \frac{MSS \times W_{max}}{RTT}$$

Packet losses are detected in 2 ways:

- **Retx Time Out (RTO):** the sender maintains a retransmission timer that is dynamically calculated: if it runs out packet is lost
- **DupACKS:** generated if packet is received multiple times or if there are OOO packets (for example 1-7 are sent but 4 is missing, then 5,6,7 send ACK(4) meaning that first OOO is 4) therefore **it is not known if dupACK is due to loss or OOO**.
  - if K>3 packet lost

What actually happens is that W is increased at every ACK, but how exactly?

Initialization: `cwnd` = 1; `ssthresh` = W_{max} (max value)
Always: W = min(`cwnd`, `rwnd`)

When congestion occurs, indicated by

K **dupACKs**: `ssthresh` = W / 2
timeout: `ssthresh` = W / 2 & `cwnd` = 1

When a new data pkt is acknowledged, increase W

if (`cwnd` ≤ `ssthresh`), SS: `cwnd` = `cwnd` + 1
if (`cwnd` > `ssthresh`), CA: `cwnd` = `cwnd` + 1 / `cwnd`

Pseudocode

Where SS lasts until the half of the latest max window size of when congestion occurred

The connection gets established in the following way:

1. host A sends syn with random initial sequence number A_seq
2. host B sends syn-ack with ACK= A_seq +1 and B_seq
3. A replies with ACK + **piggyback** that is ACK=B_seq+1 and the first packet

4.2.2) TCP Algorithms

TCP identifies application by 16-bit port numbers, moreover a **connection is uniquely identified by** (source PN, destination PN, source IP, destination IP)

Also recall the **MTU** (link layer maximum transmission unit) that imposes ancapsulation frame bit size. The IP layer has to perform fragmentation.
In TCP we have **MSS** that is the max amount of TCP data in a single ip datagram and the ip layer does fragmentation if necessary. Usually MSS=MTU-40 (ip+tcp)

Moreover TCP has communication that provides flow control, retransmission, timeouts, congestion and reciever windows, pkt reordering and duplicate discard

TCP delays ACKs in order to send ACK with another data packet, **usually one ACK every** $b(=2)$ **data PCKTS**

What are TCP features?

- **Connection Oriented:** establish connection before TX: E2E connection
- **Flow Control**
- **Reliability:** checksum, TO, retx
- **Pure App Layer Byte Stream**
- **Handling of OOO and dup**
- **Communication abstraction:** reliable ordered p2p byte streamed

TCP "Old" Tahoe

Implements:

- Slow Start
- Congestion Avoidance
- Error recovery only with time out: it RETX from first unACK pkt

TCP Tahoe (Fast Retx)

Implements:

- Slow Start
- Congestion Avoidance
- Fast Retransmit (FR)

FR works by sending the missing segment given by dupAck and then restarting the connection (SS, cwnd=1, ssthresh W/2).

TCP Reno (Fast Recovery)

Setting cwnd=1 is too aggressive, we advance cwnd by sshthrehs+ the number of dupacks recieved (does not violate pkt conservation). In Reno we enter **Fast Recovery** at the third dup ack:

When the third dupACK arrives
enter Fast Recovery:
`ssthresh` = `cwnd` / 2
RETX the missing segment **(Fast RETX)**
`cwnd` = `ssthresh` + 3 MSS
Every time a new dupACK arrives
`cwnd` = `cwnd` + 1 MSS
TX a packet if allowed by `cwnd`
When an ACK acknowledging new data arrives
`cwnd` = `ssthresh` (from first step above)
Exit Fast Recovery (resume normal operation in CA)

Pseudocode With K =3 dup acks

This means that we set the new sshtresh and we retx the missing frame. Then we advance the window by K dup akcs received. We then send the new packets (and advance windows) and receive dupacks (since we have halved W) Finally after sending all missing data we start from CA

4.2.3) TCP New Reno

This implements **Multiple RETX Fast Recovery**

```
When the Kth dupACK arrives
    recover  = lastseqno                          // highest TXed seq.no
    ssthresh = max (flightsize/2, 2 MSS)          // flightsize=lastseqno-ack_no+1
    cwnd  = ssthresh + 3 MSS                      // at least 3 OOO pkts RXed OK
    Reset RETX timer                              // to prevent timeout during fast recovery
    RETX the missing segment                      // Fast RETX
    TX a segment if allowed by cwnd
Every time a new dupACK arrives
    cwnd  = cwnd + 1 MSS                          // as 1 packet reached the receiver
    TX a packet if allowed by cwnd
When an ACK that acknowledges new data arrives
    if (ack_no>recover)
        cwnd = ssthresh                           // in step 1 above
        Exit from Fast Recovery
    else this is a "partial new ACK"              // acking "n_acked" packets
        cwnd  = cwnd - n_acked + 1
        Reset RETX timer                          // if Slow-but-steady TCP version
        RETX a packet with seq_no=ack_no
        TX a packet if allowed by cwnd
```

Pseudocode with K=3

4.2.3) TCP New Reno with Selective ACK

```
When the K-th dupACK arrives:
    recover = lastseqno;                          // highest TX seqno)
    ssthresh = max (flightsize/2, 2 MSS);         // flightsize=lastseqno-ack_no+1)
    cwnd = ssthresh + 0 MSS;                      // we don't need "+3MSS" as…)
    Reset RETX timer;                             // …pipe is updated using SACK)
    RETX the missing segment;
    pipe = lastseqno-lastackno+1;                 // # of packets in flight & not ACKed
    deflate pipe using correct RXs thanks to SACK;
    while (pipe<cwnd) {
        if (!SACK_list_empty) TX_first_missing_packet();
        else TX_new_packet();
        pipe++;
    }
Every time a new ACK (duplicate or not) arrives:
    pipe  = pipe – nacked;                        // deflate pipe using correct RXs (SACK)
    while (pipe<cwnd) {
        if (!SACK_list_empty) TX_first_missing_packet();
        else TX_new_packet();
        pipe++;
    }
When an ACK that acknowledges new data arrives (ack_no>recover):
    Exit_Fast_Recovery();
```

Pseudocode

4.2.3) TCP Header

5) Application Layer

The application layer uses a **logical connection**, it supposes that there exists a direct communication between host and client.
The application layer **only receives services** from lower protocols (Layer 4)

WWW

This is a **distributed client-server service** where each service is distributed over locations called sites that holds web pages: **file with name+address and the documento to send**. These are intercnnected via **hypertexts**

We need 4 identifiers to define a web page

- Host: (unique) IP or name of the server
- Port: 16 bit number for client server application port (HTTP uses 80)
- Path: identifies the location and the name of the file in the underlying operating system
- Protocol: abbreviation for the client-server program that we need in order to access the web page, usually HTTP

The **url** combines them together in the form **protocol://host:port/path** where :port is often omitted

This is designed to be **stateless**

HTTP

The HTTP server uses port **80** while the client a temporary port number
The Layer 4 service used is TCP

Clients send a HTTP request and host sends a HTTP response

If some of the objects in the hypertexts are located outside of the machine another TCP connection is started to retrieve them

If some are on the same machine we have 2 choices:

- **Nonpersistent HTTP:** at most one object is sent over the TCP connection. Client opens TCP request and sends request, server sends response and closes connection. This is doe until the client reads an end-of-file marker and closes the connection
- **Persistent HTTP:** Multiple objects can be sent over single TCP connection between client and server. The server can close the connection at the request of a client or if a time-out has been reached.

Cookies

WWW was designed to be stateless, but now it is required to remeber some informations about the clients

- Server collects info in the request and sends this info to the client as a cookie
- Cookie is included in next request by client
- The cookie is made and eaten by the server

COOKIE FORMAT (four fields)

FIELD ONE	FIELD TWO	FIELD THREE	FIELD FOUR
Header line of HTTP response message	Header line in HTTP *request* message	File kept on user's host, managed by user's browser	Back-end database at Web site

Cookie Format

It has 4 main purposes

- **Session Management:** for example to store login info

- **Personalization:** cookies can store user preferences/settings
- **Tracking and analytics:** advertisements
- **Shopping Cart:** remember shopping cart even when leaving site

Web Caching

A proxy can keep copies of responses to recent requests. If the request arriving to the proxy is stored in cache it will send it back, otherwise it will forward to real server that will reply. This can lower response times

E-Mail

In mails the **response is non mandatory**, this is considered a **one way transaction**
To ensure that the receiver has received the mail intermediate servers are used. These servers store the message until B is reachable

Each user has a User Agent (UA) that handles composition, reading replying, forwarding and also the local mailbox. It can be **Command driven or GUI based**

The **Message Transfer Agent (MAA)** uses **Simple Mail Transfer Protocol (SMTP)** is used to forward the mail from the UA to the mail Server, and again from the first to the last mail server

Message Access Agent (MAA) is used for the receiver to retrieve the message from the mail server to the UA

SMTP

The mail transfer process is divided in 3 phases:

Connection Establishment:

- client establishes connection to TCP port 25
- Server sends code 220 (service ready) or 421 (service not available)
- If 220 is received it sends HELO message to identify itself using domain + address
- Server responds with code 250 (request command completed)

Message Transfer

- client sends MAIL FROM message with mail of sender
- Server responds with code 250
- Client sends RCPT TO and includes mail of recipient
- Server response 250
- Client sends DATA to initialize transfer
- Server response 354 (start mail input)
- client sends the contents of the message in consecutive lines. Each line is terminated by a two-character end-of-line token. The message is terminated by a line containing just one period.
- Server response 250

Connection Termination

- client sends QUIT command
- server responds with code 221

MAA

We have may types

Post Office Protocol (POP) 3

Clients pen connection on TCP 110
Sends user + passw

User can list and retrieve messages

It cannot:

- allow user to organize mail on server
- different folders
- partially check mail before download

IMAP4

POP3 + some functions like:

- check mail header before download
- search mail for specific string
- partial download
- create rename or delete mailboxes

Multipurpose Internet Mail Extension (MIME)

Mails use ASCII format. To send symbols or files this is a translator from file to ascii and vice versa

5.2) Domain Name System (DNS)

Humans don' remember numbers easily, therefore we use DNS to map a name to an IP address. These maps are distributed among many (13) computer in the world and the host that needs mapping can contact the closest computer to get the right IP.

Names are unique (1:1) and **hierarchical** (max 128 layers) and can be found using an inverted tree structure

The 13 DNS servers belong to a commercial entity called ICANN where you can also register the IPs (.com, .org, .edu). Moreover there are also **local DNS** servers belonging to ISPs. The ip of the DNS servers must be known (duh)

When the DNS request is sent it goes first to the local DNS server, if it is not cached it goes to one of the 13 DNS servers and if they also didn't cache it it goes to the real dns host. If the dns request reaches the host it gives an authoritative answer since it responds with the "real truth". In fact the caches on the DNS have a TTL but could be wrong sooner

DNS attacks:

Attack Type	Target	Goal
1. Compromised machine	Local DNS cache	Fully control DNS resolution
2. Spoofed replies	User device	Trick into caching fake response
3. Cache poisoning	Local resolver (DNS server)	Affect multiple users at once
4. Malicious DNS server	DNS infrastructure	Serve fake or malicious DNS data

6) Security

Let's first define some security goals:

Security Goal	Definition
Confidentiality/Anonimity	Information is available only to the intended receiver.
Integrity	Information is received as sent. If something is changed it should be possible to detect what is changed and who changed it
Availability	It is always possible to identify who did any action
Privacy/Controlled Access	Information is used but disclosed only to authorized entities

Also the threats:

Attack/Threat	Purpose
Eavesdropping	Learning confidential info
Traffic Analysis	Learning origin, destination, length but not content
Forgery	Building fake message pretending it was sent by someone else
Masquerade	Claiming to be someone else
Repudiation	Denying to having sent/received a message
Profiling	Gathering info about a single user
Fingerprinting	Identifying user associated to message
Jamming	Causing interference to a communication

A layer N security protocol is a series of mechanisms that offer some security services at the N-th layer **and above**

We will study network attacks. These access and modify internet traffic using:

- **Sniffing:** intercept traffic to read header and data (not necessarily bad)
- **Spoofing:** manipulation of the traffic (usually non-legitimate action)

These are used to launch both DOS and MITM attacks.

- **Denial Of Service (DOS):** interrupts availability of service. Historically a **smurf** attack was used: the attacker sends a broadcast ICMP echo request with the victim's IP destination address that get's overwhelmed by echo replies
- **Man In The Middle (MITM):** it intercepts and changes packet content (ARP reg or DNS cache). For example by changing the routing information of the victim to forward the packet to a malicious router. To do so the malicious router should disable ICMP **redirect** messages and continuously send ARP updates to the victim that otherwise discovers the real router (that has a better route). After spoofing the packets to forward it should also modify the checksum of IP and TCP protocols.

There also exists a **sinkhole attack** where using advertisement packets a malicious router is chosen by many neighbors for the optimal route. Also a **wormhole attack** it creates a malicious link between two malicious devices that advertise the link as low latency and all the data get's sent through here. **Ping of Death** by sending a oversized ping packet the buffer of the victim might get overflown and crash or freeze or reboot. **Distributed Denial of**

Service (DDoS) to cause a DOF by generating many requests through a botnet to a victim. These bots overwhelm the site that then legitimate users can't access.

6.1) Cryptography

This is the study of how to encipher and decipher messages. There are two main types od cryptography:

- **Asymmetric/Public:** uses different keys for encryption and decryption. These algorithms are easy to compute. However the private key is infeasible to compute and the cipher text is can't be computed without the private key
- **Symmetric/Private/Secret:** same key for encryption and decryption but an additional mechanism is required to distribute the private key

Hashing

A hash function is any function that can be used to **map data of arbitrary size to data of fixed size**.
A **one way hashing function** satisfies the following two:

- **One way property:** given the hashed data h it should be difficult to obtain m such that $\text{hash}(m) = h$.
- **Collision Resistance:** it should be difficult to have $\text{hash}(m_1) = texthash(m_2)$

A possible risk is a brute force attack: if the possible data is small I can brute force all data hashes until I find the correct hash.

Two popular hashing functions are the **Message digest (MD) and Secure Hash Algorithm (SHA) series** MD2,MD4 and SHA0, SHA1 were broken. SHA2 is the most used even though SHA3 exists.

MD5, SHA1 and SHA2 use the same method called **Merkle-Damgard construction:**

- Data is divided into even sized blocks (+padding for last block)
- Each block is fed to a compressor block (each standard uses a different function)
- The output of the last block is the hash

Even a small change is enough to completely alter the hash.
Passwords are saved as hashes and the saved hash is compared to the has of the password the user provides for auth.

Asymmetric Cryptography

Here the public key is known to everyone but the private key is known only to B. Moreover the private key is distributed and the link between the owner and key should be certified and the link between public and private key should be trusted. For this we use **Digital Certificates.**

Private key A and Public key A do the mathematical inverse function. This is secure only one way and thus it is used for authetntication and not for data delivery

Digital Certificates are managed by third party Certificate Authority (AC). To obtain a certificate each entity provides its identity and key to the Ca that issues the certificate with:

- Name of entity
- Certificate value
- CA that issued it
- It is signed with CA's private key and anyone with public key can verify it

The last step is a **digital signature** it works in 2 steps:

1. Signing: hash the message and encrypt the hash with the private key. Send message with signature

2. Verification: The receiver hashes the message and decrypts the signature. If the two match it is verified

Symmetric/Private Key Cryptography

This has the same key for both encoding and decoding, mathematically encryption and decryption are the mathematical inverse. It is unfeasable to decrypt without the key

The first encryption comes from Julius Caesar as a type of shift cipher, from here the name **caesar cipher.** The 26 letters of the alphabet are shifted in the encoded message:

$$E_n(x) = (x + n) \mod 26$$
$$D_n(x) = (x - n) \mod 26$$

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W

```
Plaintext:  THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
Ciphertext: QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
```

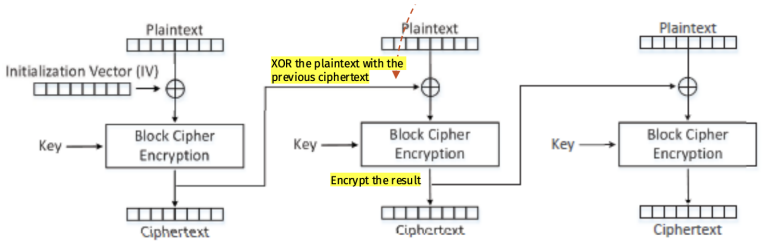
Example

many examples through history show how decrypting a message has helped in some situations.

in 1976 the **Data Encryption Standard (DES)** was introduced as a encryption with block size of 64 bits (56+8 of error). It is easy to brute force and is now replaced by **Advanced Encryption Standard (AES)**. It has bigger blocks, but this is not secure since the same blocks will be encoded in the same ways. Therefore it must be possible to not have a 1:1 corrispondence

Electronic CodeBook (ECB): similar to DES or AES but has a key to encode. Highly unsafe

Cipher Block Chaining (CBC): Each block is XOR-ed with the previous block. Moreover there is a random **initialization vector (IV)** that ensures more security since different IV's produce different outputs. However it is sequential and can't be done in parallel



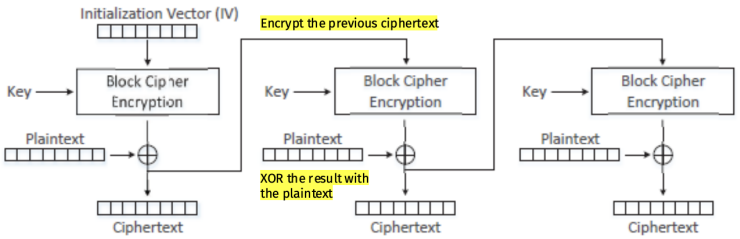
CBC

$$C_i = E_k(P_i \oplus C_{i-1})$$

with:

- C ciphertext (C_0 is initialization vector)
- P plain text
- E encription function with key k

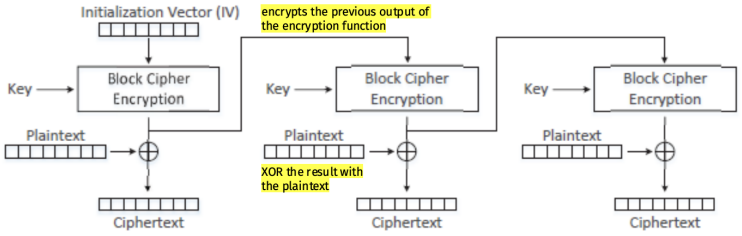
Cipher FeedBack (CFB): Each block is XOR-ed with the encoded previous block. It can start encryption without full message but an error is propagated until the output



CFB

$$C_i = P_i \oplus (E_k C_{i-1})$$

Output FeedBack (OFB): Xor with previous output before it is XOR-ed.



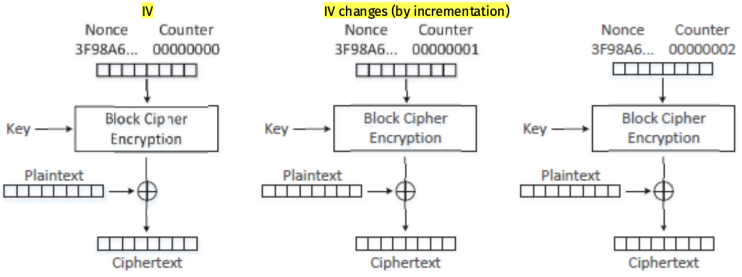
OFB

$$C_i = P_i \oplus (E_k O_{i-1})$$

with:

- O the block cipher encryption output

CounTeR (CTR): It does not have correlation with the previous block. The IV is increased at each block and therefore is parallelizable with no error propagation



CTR

$$C_i = P_i \oplus E_k(CTR_i)$$

with:

- CTR counter value for block i

Key Exchange

Keys must be securely exchanged. It is possible to do with a public-key cryptography, but a crypto system might bee too much overhead for just a key exchange.

We use the **Diffie-Hellman Key Exchange:**

- **Cyclic Group:** A big prime number p is chosen
- **Generator:** a small prime number g
- A and B pick two random positive integers $x, y < p$
- A sends $L = g^x \mod p$ and B sends $M = g^y \mod p$
- A computes $K = M^x \mod p$ and B computes $K' = L^y \mod p$
- Bothe return the same key $K = g^{xy} \mod p$
- Since $g^x \mod p$ is a NP-hard problem to compute x (and y) then the key is securely transmitted

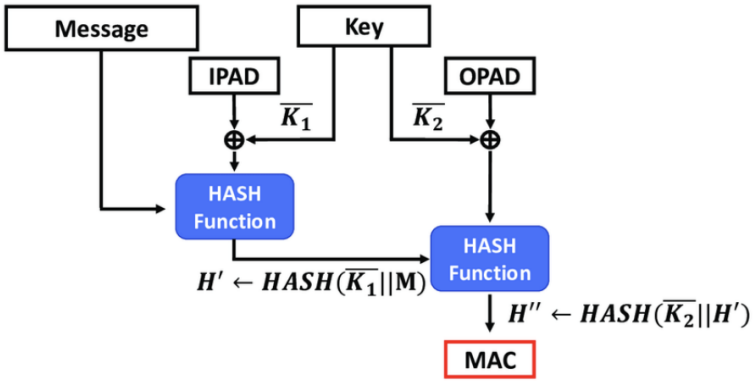
Aspect	Symmetric cryptography	Asymmetric cryptography
Key usage	Single shared key for encryption and decryption.	Uses a pair of public and private keys.
Speed	Faster because of simpler algorithms and shorter key lengths.	Slower due to more complex algorithms and longer key lengths.
Key management	Requires secure distribution of the shared key to all parties.	No need to share private key; public key can be freely distributed.
Security	Less secure if the shared key is intercepted or leaked.	More secure as private key is never shared.
Computational overhead	Low computational overhead, efficient for large-scale encryption.	High computational overhead, more suited for initial secure handshakes.

Recap

Authentication

The **Message Authentication Code (MAC)** is used to detect if a file has been changed. It uses a known function and secret key to create a short appendix based on the plaintext to append to the message. If the message is changed and the append doesn't match it is discarded. It should be infeasable to compute the tag without the key. Usually hashing is used. It is called **Key Hash Mac (HMAC)**, it works in the following way:

- B size of block used by H (64 bits)
- Key k of variable size (padded to size of B)
- Two hashes are used and combined together Inner hash and Outer hash have a fixed value padded B times, from here the names ipad/opad
- Xor key and message
- Compute hash of result



Diagram

Aspect	MAC	Digital Signature
Based on	Symmetric key (shared secret)	Asymmetric keys (private/public)
Key ownership	Shared between sender and receiver	Private key held by sender, public key held by everyone
How it works	Sender and receiver use the same secret key to generate/verify the MAC	Sender uses **private key** to sign; receiver uses **public key** to verify
Main Purpose	Integrity + authentication	Integrity + authentication + **non-repudiation** (sender cannot deny having sent it)

Recap

6.2) Network Security Applications

Usually security is applied in only two layers:

- Application: E2E protection can be guaranteed and doesn't add cost to lower layers
- Transport/Network: shared by multiple applications

The **Transport Layer Security (TLS)** is designed to work with TCP for:

- authentication endpoints
- exchanging confidential data
- authenticating messages

It has 2 primary components:

- **Handshake:** used to negotiate cryptographic modes and parameters and establish the shared key
- **Record:** uses parameters of handshake to protect traffic

The **Datagram Transport Layer Security (DTLS)** is designed to work with UDP
It has the problem to be limited by underlying protocols of limited packet size and therefore implements a compression mechanism
It creates point-to-point secure association not compatible with multicast IP communications
It provides 3 security modes:

- PreSharedKey: device store symmetric pre-share keys
- RawPublicKey: devices owna private-public key pair without certificate
- Certificare: devices store X.509 certificate

The **Internet Protocol Security (IPsec)** works on the network layer.
It provides confidentiality, integrity, data-origin auth and protection against reply attacks
It includes 2 principal Protocols:

- Authentication Header (AH): source auth and data integrity
- Encapsulation Security Payload (ESP): provides confidentiality
  There are 2 types of packets forms:
- Tunnel mode: ip pck is encapsulated as a payload of new IP pck. A new header is added. It protects traffic between different nets and simplifies key exchange procedure
- Transport mode: retains original IP header (less secure) but is used for E2E communication between two secure hosts

These datagrams are sent of there is a secure logical connection called **Security Association (SA)** that must be established in both directions and stores type of encryption and integrity check, ...

6.3) Firewall

The firewall stops unauthorized traffic flowing from one network to another. It should:

- Pass all traffic between two trusted zones
- Pass only authorized traffic
- be immune to penetration

It is configured by an admin through a set of policies and should provide:

- User Control: rules based on role of user inside firewall
- Service Control: rules based on type of service, usually based on sockets
- Direction Control: based on direction of flow

It has 3 possible outcomes:

- Accepted
- Denied
- Rejected: same as Denied but sends ICMP to source

It uses headers and data from the packets to obtain infos. Moreover encryption might prevent firewalls to access certain information and thus discard the packets. Usually firewalls are implemented at the network layer so any upper layer encryption passes, while lower layers (DLL) don't pass.

Types of Firewalls

A firewall is **stateless** as it analyzes each packet individually. Firewalls might have different goals:

- Ingress filtering: safeguard inner network by inspecting incoming traffic
- Egress filtering: prevent users to reach certain services or send certain data by inspecting outgoing traffic

Packet Filtering controls data based on source/destination IP, transport protocol, ports, flags and network interfaces. It will then take one of the following **actions:** drop/pass the packet and then eventually log an error to the sender or log the packet to the user

Example:

Rule	Direction	Src. Addr.	Dest. Addr.	Protocol	Src. Port	Dest. Port	ACK	Action
A	Inbound	External	Internal	TCP		25		Permit
B	Outbound	Internal	External	TCP		>1023		Permit
C	Outbound	Internal	External	TCP		25		Permit
D	Inbound	External	Internal	TCP		>1023		Permit
E	Either	Any	Any	Any		Any		Deny

Example

This passes SMTP mail (port 25) but the sender might send from a port > 1023. If someone uses spoofing to enter the firewall by pretending it is from an internal network it is blocked. The service X11 is on port 6000 and thus it's traffic is allowed in and out.

Rule	Direction	Src. Addr.	Dest. Addr.	Protocol	Src. Port	Dest. Port	ACK	Action
A	Inbound	External	Internal	TCP	>1023	25		Permit
B	Outbound	Internal	External	TCP	25	>1023		Permit
C	Outbound	Internal	External	TCP	>1023	25		Permit
D	Inbound	External	Internal	TCP	25	>1023		Permit
E	Either	Any	Any	Any	Any	Any		Deny

Updated Example

This is now fixed since rule A blocks traffic incoming traffic that doesn't go to port 25. By also specifying the ack of B and D to be "YES" it blocks new TCP connections

Stateful Firewalls monitors the interaction of the connection until it is closed. It can therefore decide actions based on context. They can limit the range of TCP/UDP sessions based on what is required $\Longrightarrow$ less spoofing. This checks protocol state, allow valid sequences and protect from spoofing. They track sessions by monitoring packest and tag them as:

- New: first packet of connection
- Established: packet of existing connection
- Related: related to an existing connection
- Invalid: invalid header values

Therefore it can also prevent DOS, limit max amounts of sessions, apply timeouts to idle sessions

Example:

Rule	Direction	Src. Addr.	Dest. Addr.	Protocol	Src. Port	Dest. Port	ACK	Action
A1	Inbound	External	Webserver	TCP	>1023	80	Any	Permit
A2	Outbound	Webserv.	External	TCP	80	>1023	Yes	Permit
B1	Outbound	Internal	External	TCP	>1023	80	Any	Permit
B2	Inbound	External	Internal	TCP	80	>1023	Yes	Permit
C1	Outbound	Internal	External	UDP	>1023	53	-	Permit
C2	Inbound	External	Internal	UDP	53	>1023	-	Permit
D	Either	Any	Any	Any	Any	Any	Any	Deny

Example

This allows HTTP (TCP 80) traffic by external hosts and allows internal hosts to initiate HTTP or DNS (UDP 53) while denying all other communications.

Rule	Direction	Source Address	Destination Address	Protocol	Source Port	Dest. Port	State	Action
A	Inbound	External	Webserver	TCP	>1023	80	New	Permit
B	Outbound	Internal	External	TCP	>1023	80	New	Permit
C	Outbound	Internal	External	UDP	>1023	53	New	Permit
D	Either	Any	Any	Any	Any	Any	Established	Permit
E	Either	Any	Any	Any	Any	Any	Any	Deny

Stateful Rules

For HTTP and DNS it allows new connection. Already established connections are also permitted

Proxy Firewall inspects traffic on the application layer and acts as an intermediary by impersonating the recipient. It is slow

6.4) Linux Firewall

Linux offers a framework called **netfilter** that uses hooks to analyze/manipulate packets and return a verdict on them
There are 5 hooks:

- NF_INET_PRE_ROUTING: all incoming packets hit this hook before any routing decision is made
- NF_INET_FORWARD and NF_INET_LOCAL_IN: decide if the packet is to route or if it is on the local path
- NF_INET_LOCAL_OUT: if the packet is generated by this host it hits this hook at layer 3
- NF_INET_POST_ROUTING: when packet is going out of host

There are 5 possible verdicts

Verdict	Description
NF_ACCEPT	The pkt can continue its journey
NF_DROP	Discard the pkt
NF_QUEUE	It can be used to queue the pkt in the user space
NF_STOLEN	netfilter should forget about this pkt. It is passed to some other module for processing (typically fragmented pkts are treated in this way so that they can be analyzed together)
NF_REPEAT	Request to repeat the module

Verdicts

Xtables is a firewall based on **netfilter**. The user-space progra,m to configure the firewall is **iptables**. Moreover **nf_conntrack** buildt on top of **netfilter** allows to store information about the session.